

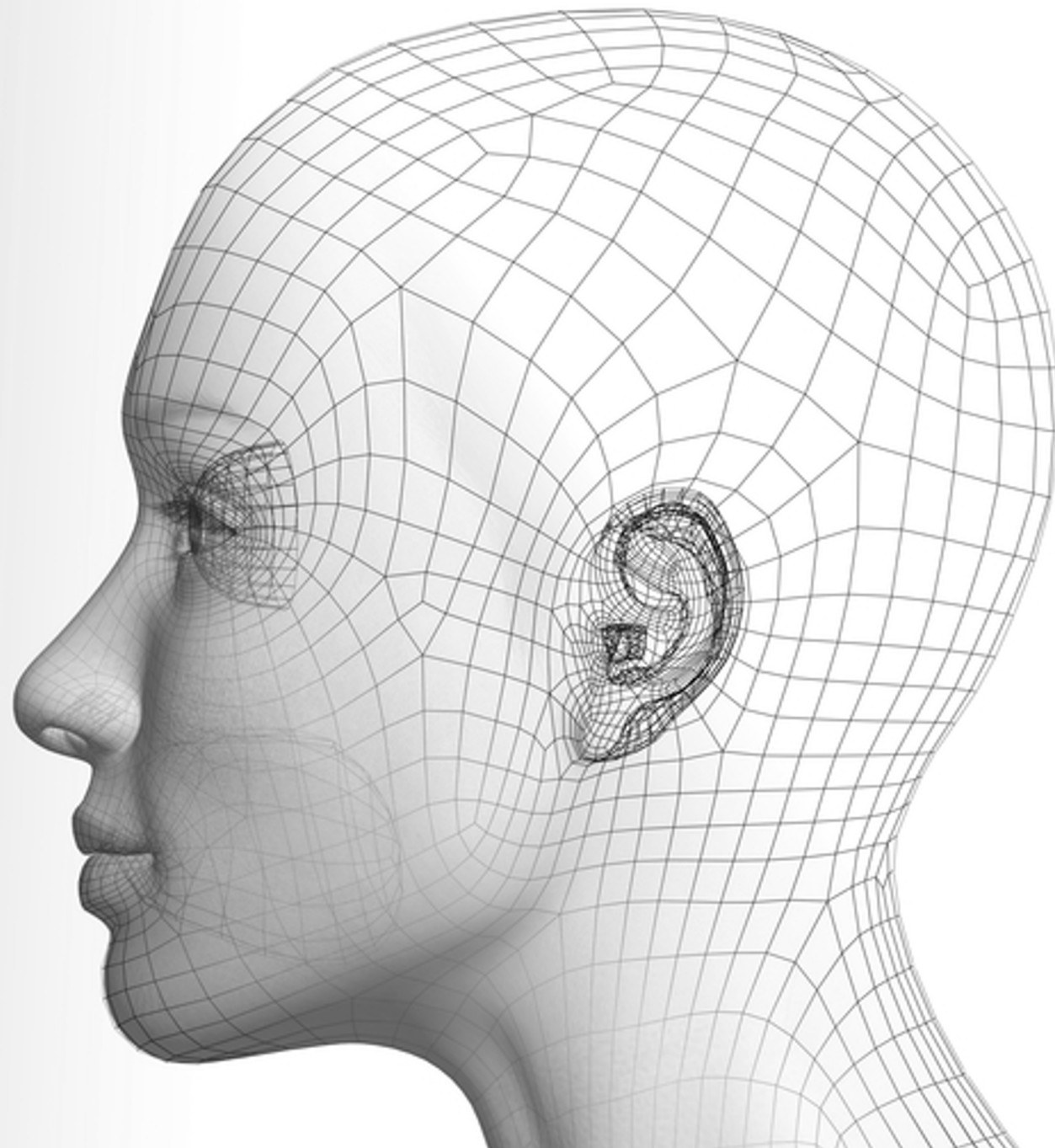
Transformers for Computer Vision

Chen-Change Loy

吕健勤

<https://www.mmlab-ntu.com/>

<https://twitter.com/ccloy>

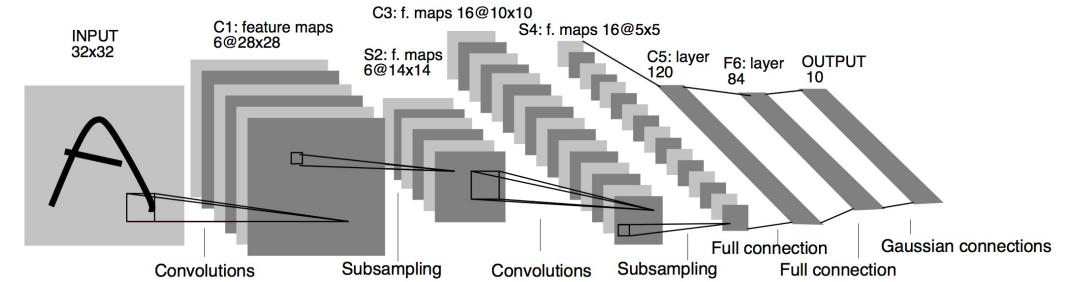


Outline

- Attention
- Sequence-to-Sequence with RNNs and Attention
- Transformers
- Extra – Vision Transformers

Background

- Convolutional neural networks (CNN) has been dominating
 - Greater scale
 - More extensive connections
 - More sophisticated forms of convolution



- Transformers
 - Competitive alternative to CNN
 - Generally found to perform best in settings with large amounts of training data
 - Enable multi-modality learning

Credits

- A nice blog by Lilian Weng:
 - <https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html>
 - <https://lilianweng.github.io/lil-log/2020/04/07/the-transformer-family.html>
- The illustrated Transformer by Jay Alammar
 - <https://jalammar.github.io/illustrated-transformer/>

Attention

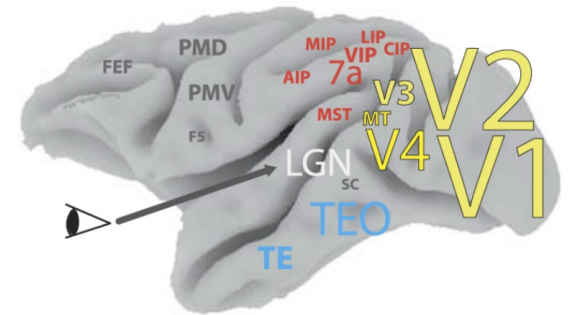


"Where's Waldo?"

Visual Attention

Scientists divided into two camps

- In the first model, the spotlight of attention would track across the page, checking each detail against a mental image of Waldo's red stocking cap and striped shirt.
- In the second model, the color red and stocking-cap shapes would gradually come to the foreground and other shapes and colors would recede.
- The study by MIT showed that both processes are going on in the same chunk of the brain and in the same neurons
- Midregion of the V4 visual cortex known to be important to attention (V4 is tuned for object features of intermediate complexity, like simple geometric shapes)



Attention

Encyclopedia of Neuroscience

2009 Edition | Editors: Marc D. Binder, Nobutaka Hirokawa, Uwe Windhorst

The term “**visual attention**” refers to a set of cognitive operations that mediate the selection of relevant and the filtering out of irrelevant information from cluttered visual scenes.

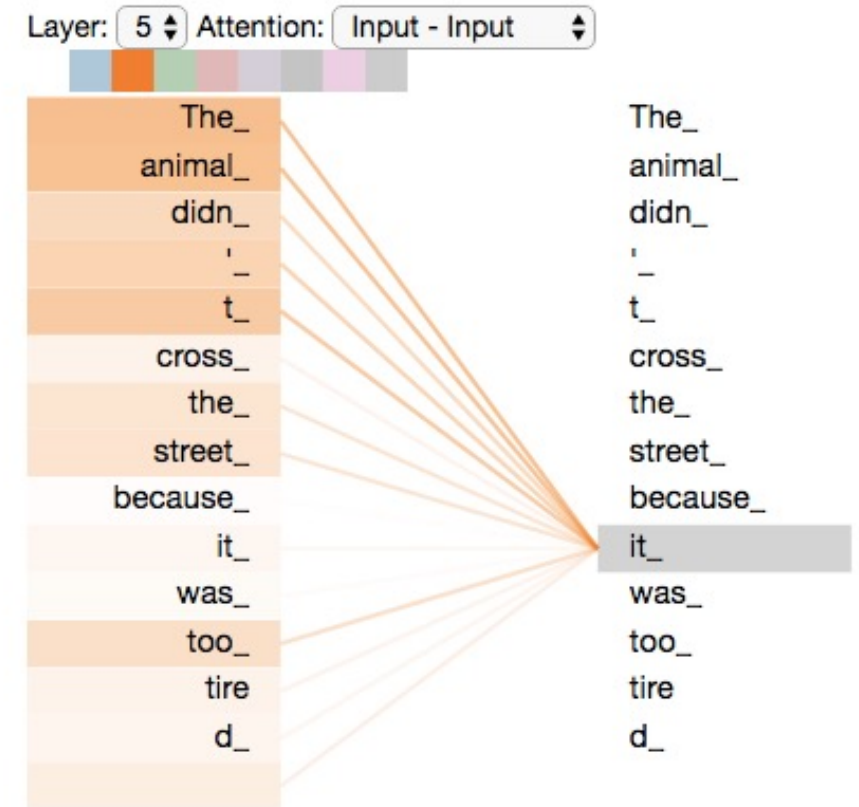
- Reduce complexity
- Resource saving

Attention

Attention is a mechanism that a model can learn to make predictions by selectively attending to a given set of data.

The amount of attention is **quantified by learned weights** and thus the output is usually formed as a **weighted average**.

Self-attention is a type of attention mechanism where the model makes prediction for one part of a data sample using other parts of the observation about the same sample.



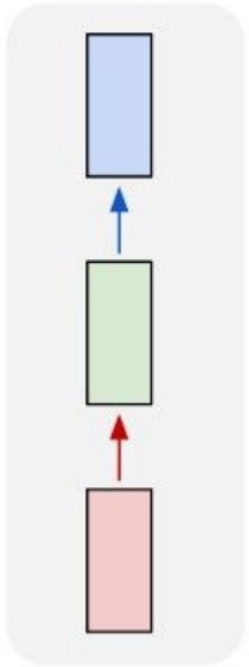
What does “it” in this sentence refer to? Is it referring to the street or to the animal?

When the model is processing the word “it”, self-attention allows it to associate “it” with “animal”.

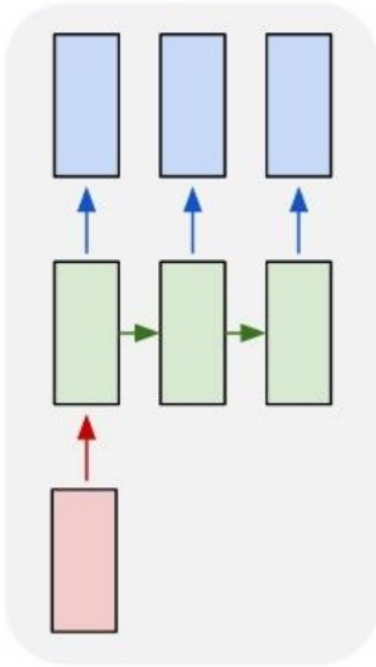
Sequence-to-Sequence with RNNs and Attention

Input-output scenarios

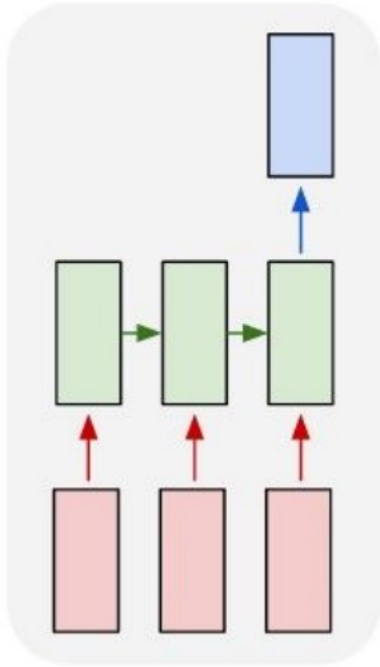
one to one



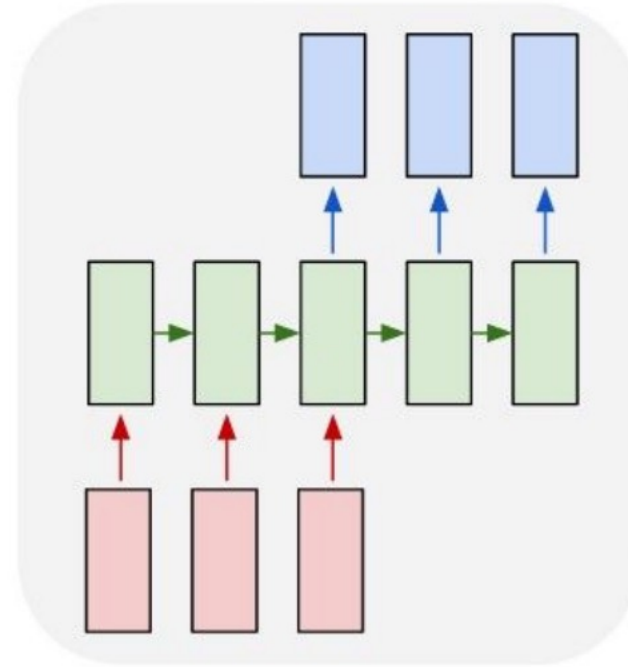
one to many



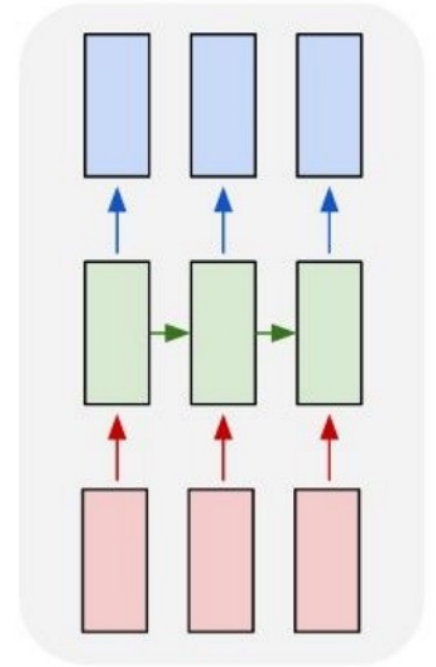
many to one



many to many



many to many



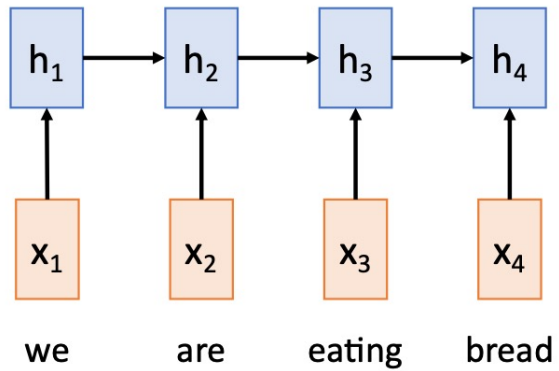
e.g. **Machine Translation**
seq of words -> seq of words

Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$



Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

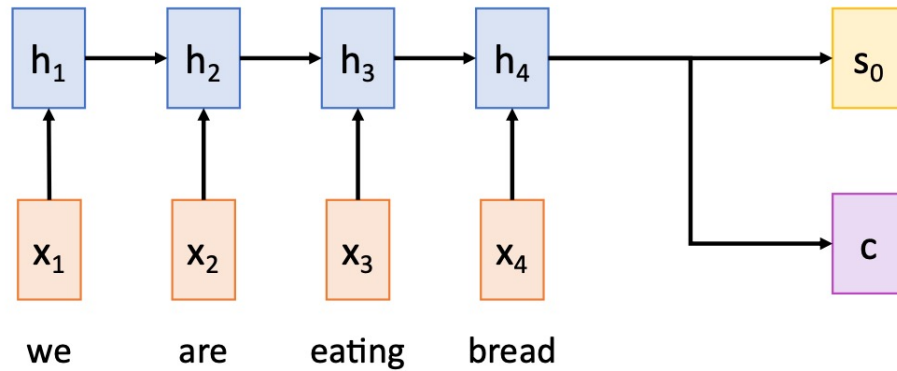
Output: Sequence $y_1, \dots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)

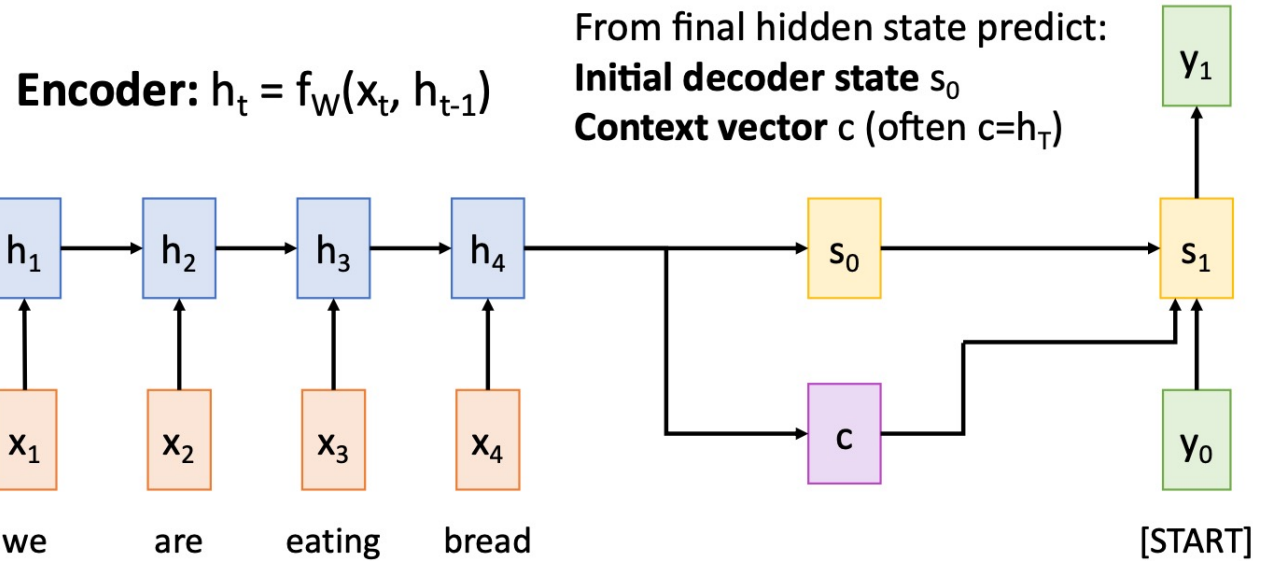


Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

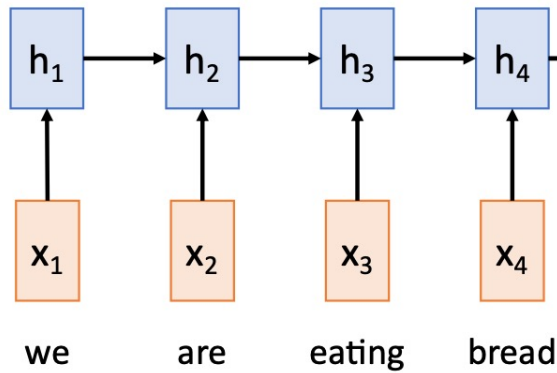


Sequence-to-Sequence with RNNs

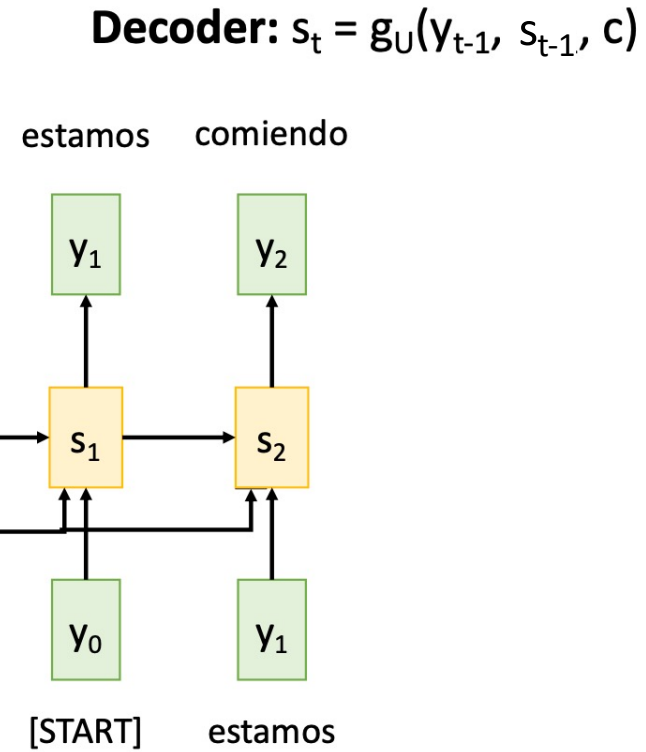
Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

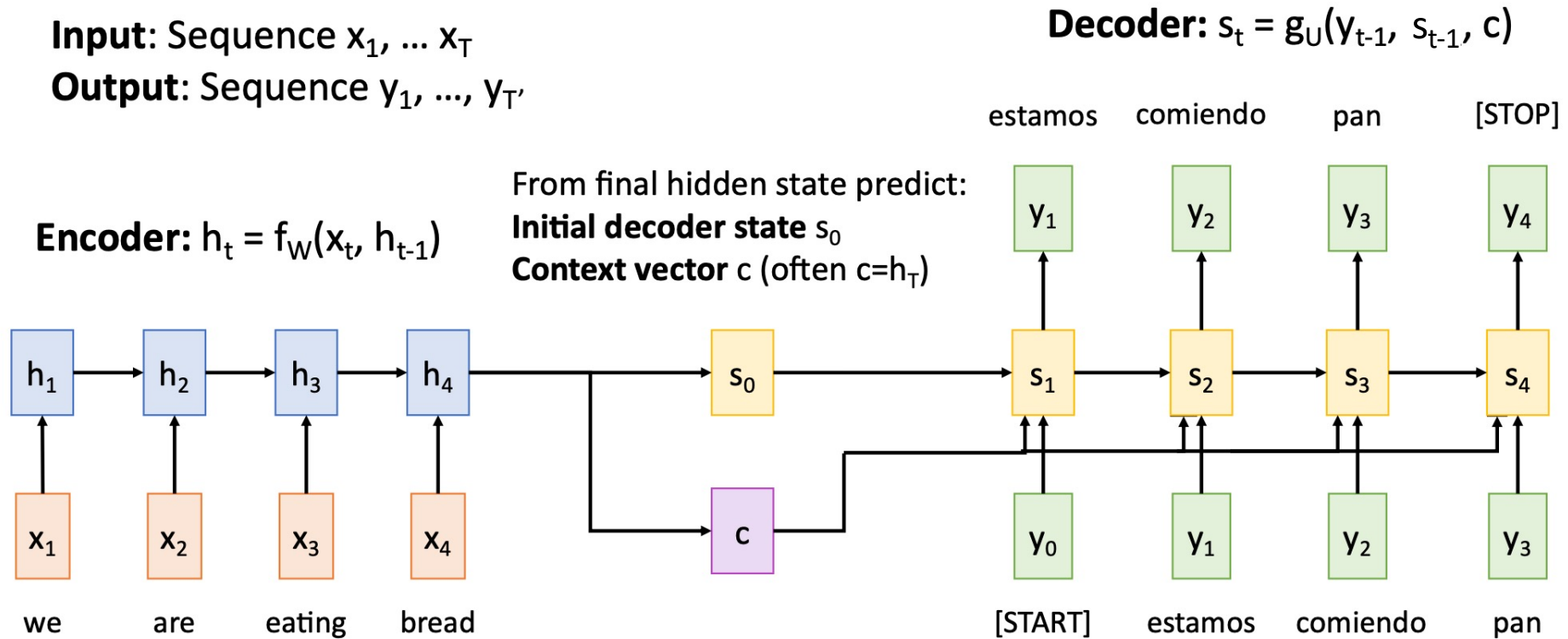
Encoder: $h_t = f_W(x_t, h_{t-1})$



From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c=h_T$)



Sequence-to-Sequence with RNNs

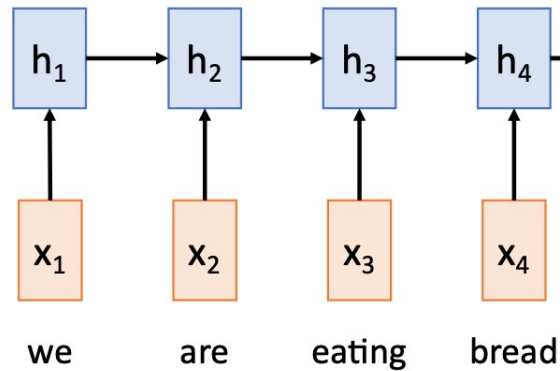


Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

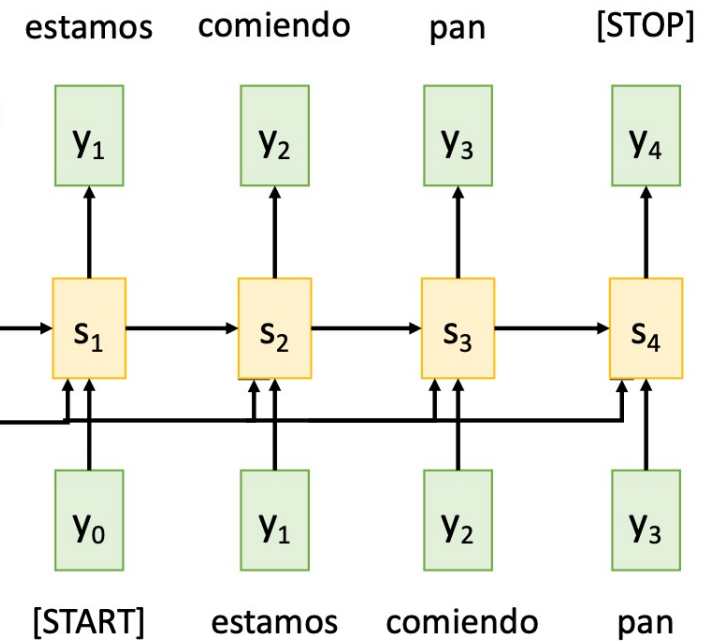
Output: Sequence $y_1, \dots, y_T$

Encoder: $h_t = f_W(x_t, h_{t-1})$



From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c=h_T$)

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$



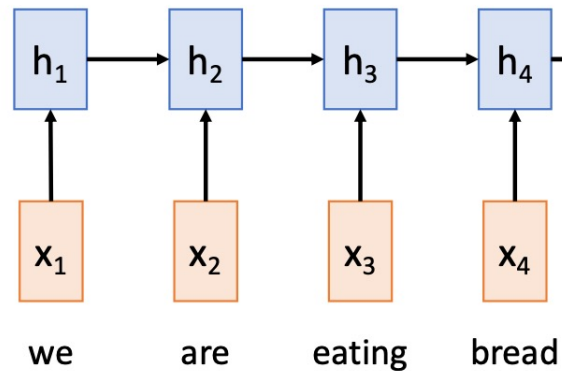
Context vector summarizes all the information need for the decoder

Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

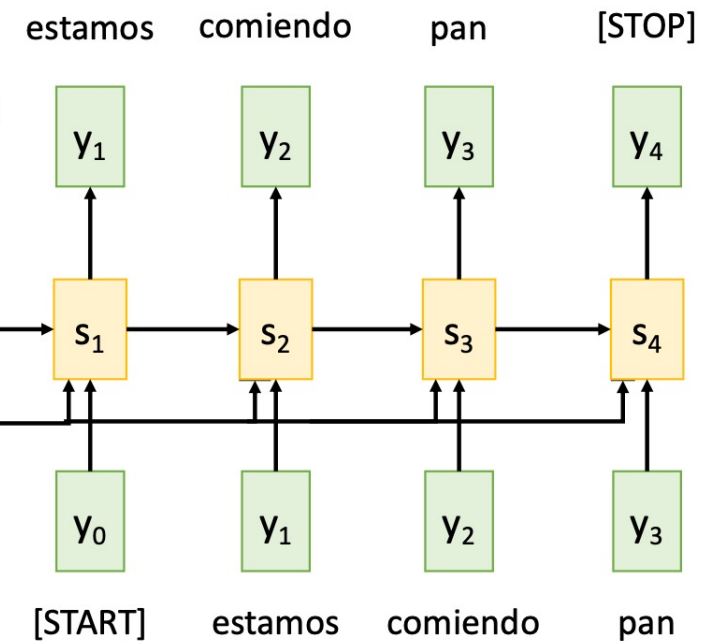
Output: Sequence $y_1, \dots, y_T$

Encoder: $h_t = f_W(x_t, h_{t-1})$



From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c=h_T$)

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$



Problem: Input sequence bottlenecked through fixed-sized vector. What if $T=1000$?

Idea: use new context vector at each step of decoder!

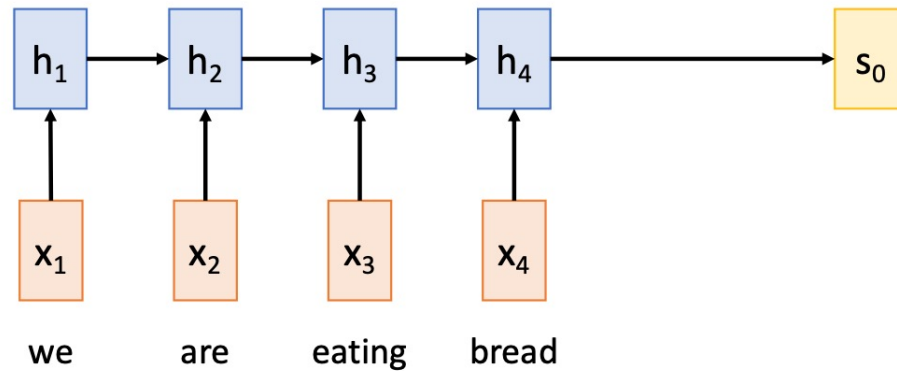
Sequence-to-Sequence with RNNs and Attention

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state:
Initial decoder state s_0



Sequence-to-Sequence with RNNs and Attention

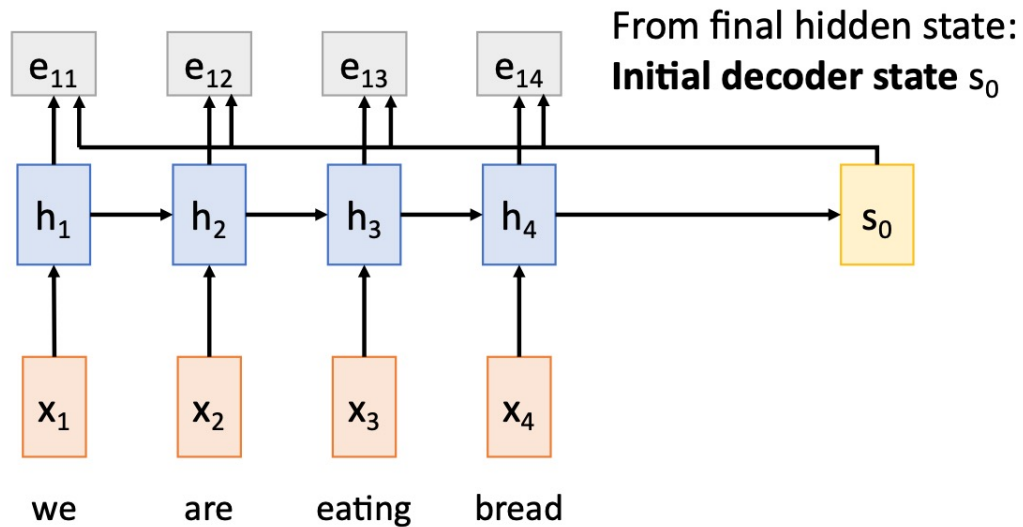
To predict how much we should attend to each hidden state of the encoder given the current hidden state of the decoder

Compute (scalar) **alignment scores**

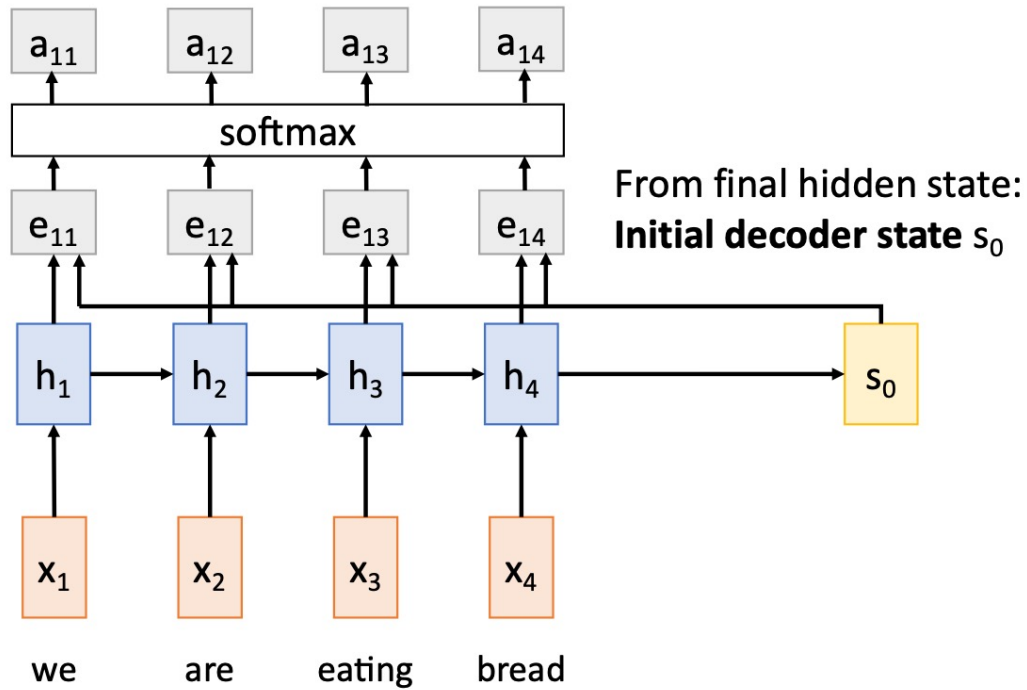
$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad (f_{\text{att}} \text{ is an MLP})$$



scalar



Sequence-to-Sequence with RNNs and Attention



Compute (scalar) **alignment scores**

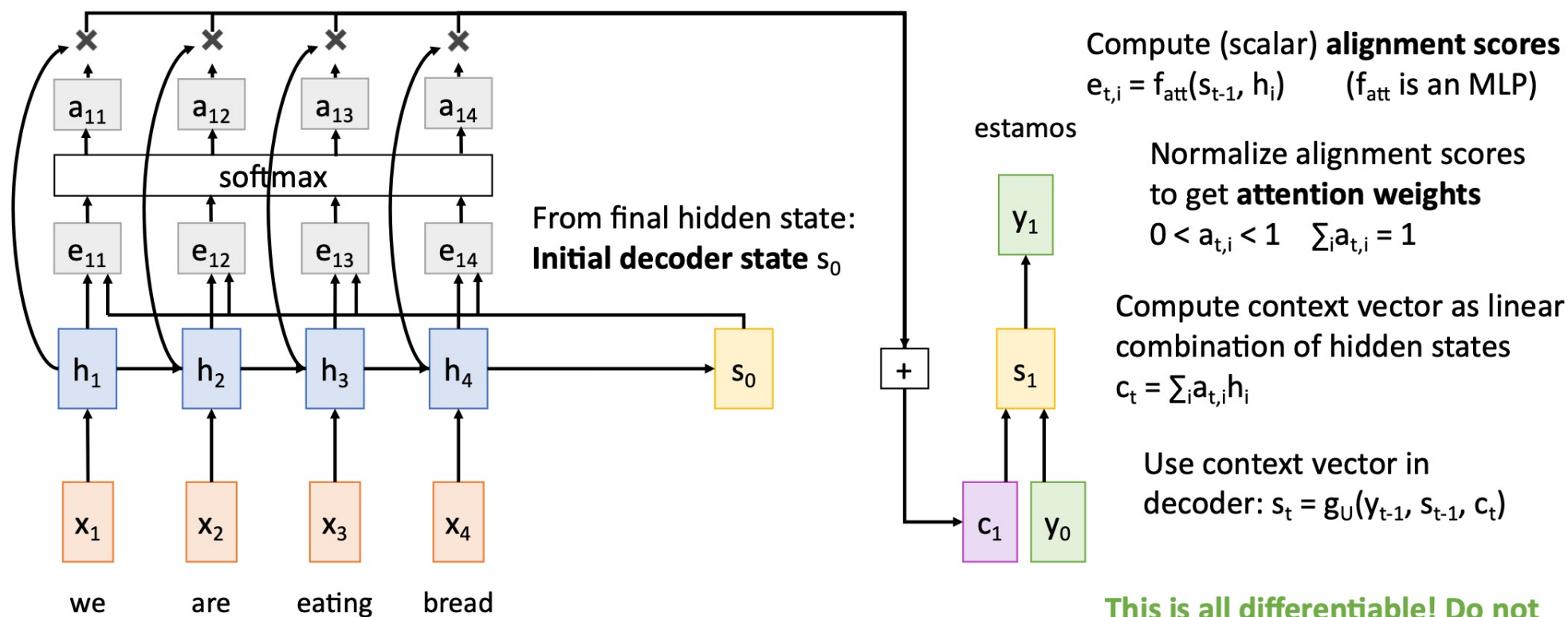
$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad (f_{\text{att}} \text{ is an MLP})$$

Normalize alignment scores

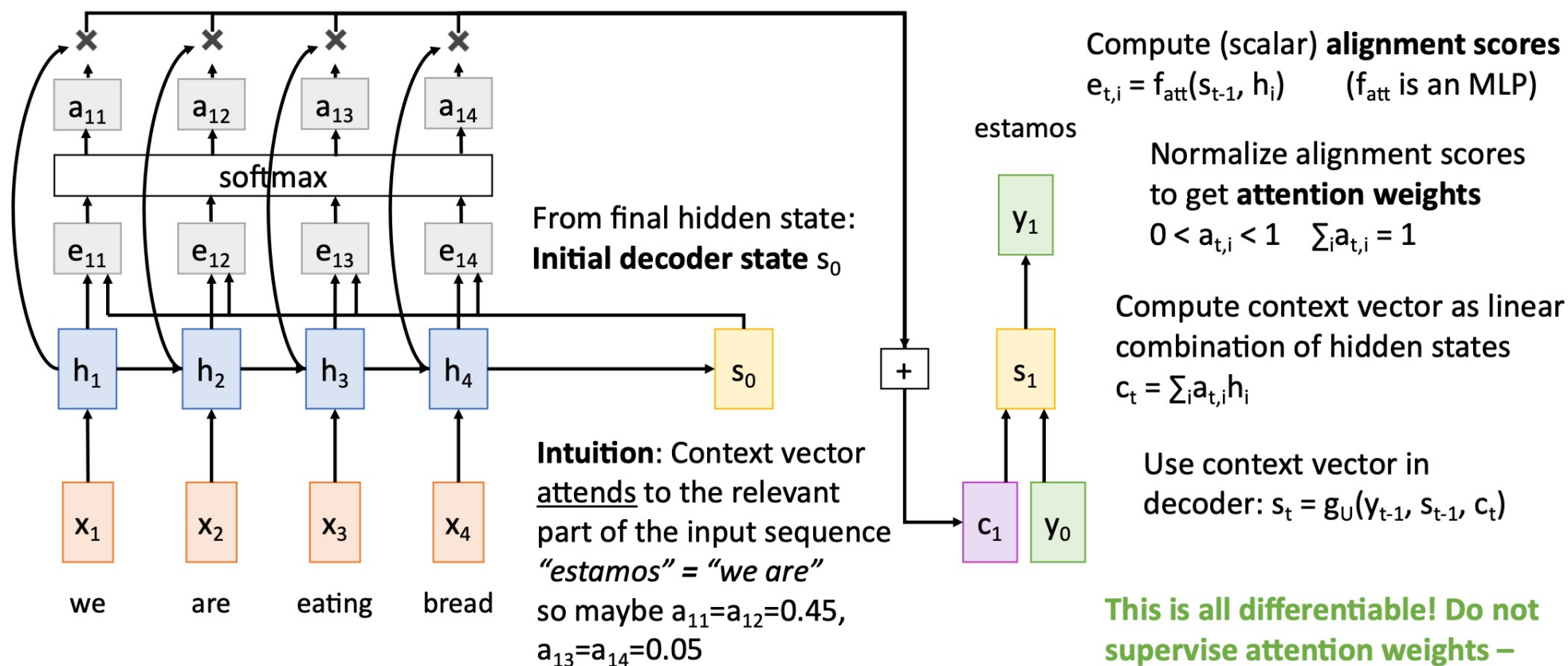
to get **attention weights**

$$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$$

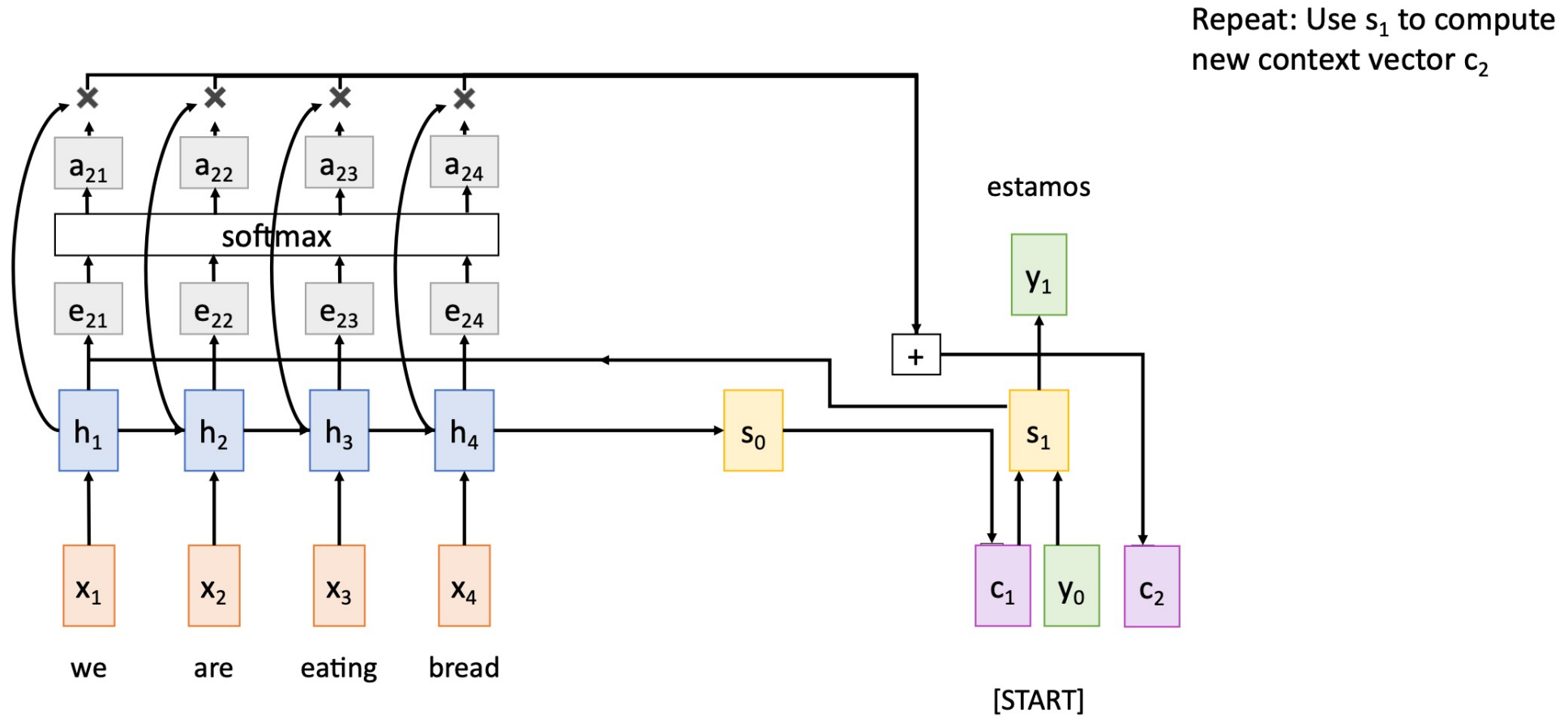
Sequence-to-Sequence with RNNs and Attention



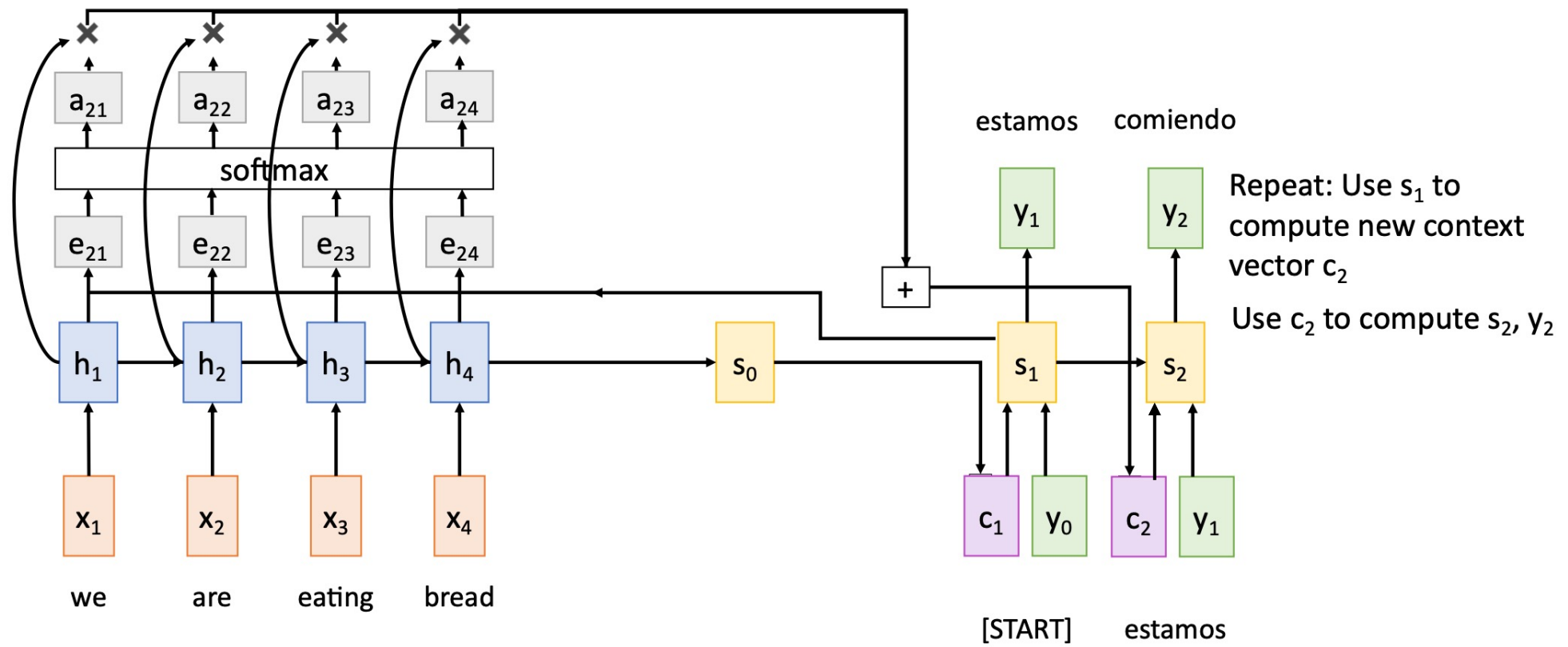
Sequence-to-Sequence with RNNs and Attention



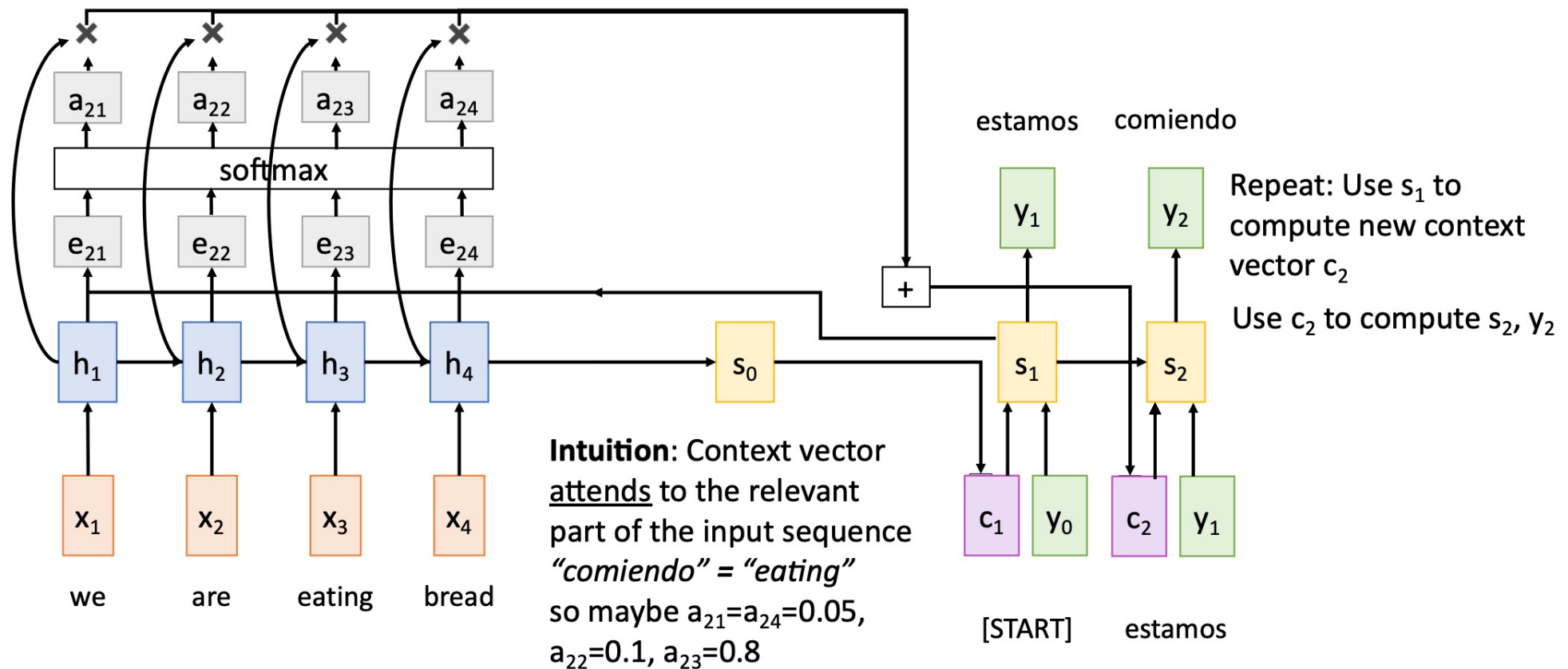
Sequence-to-Sequence with RNNs and Attention



Sequence-to-Sequence with RNNs and Attention



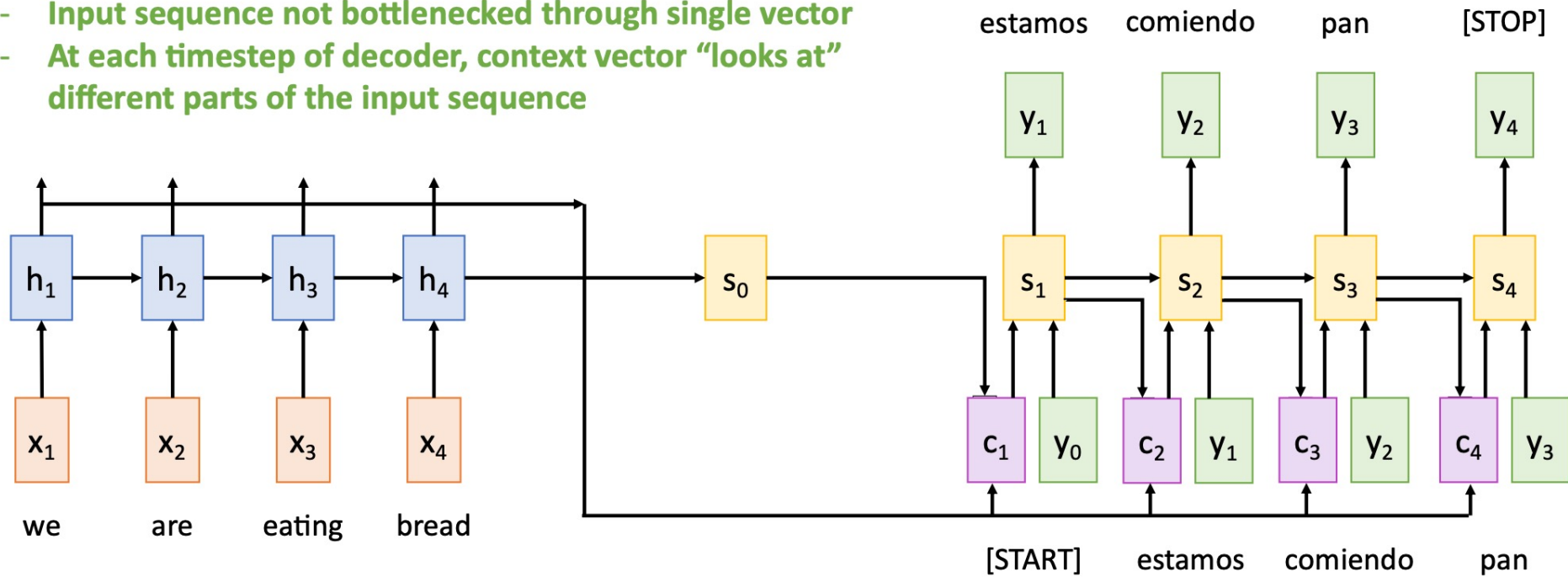
Sequence-to-Sequence with RNNs and Attention



Sequence-to-Sequence with RNNs and Attention

Use a different context vector in each timestep of decoder

- Input sequence not bottlenecked through single vector
- At each timestep of decoder, context vector “looks at” different parts of the input sequence



Sequence-to-Sequence with RNNs and Attention

Example: English to French translation

Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L'accord sur la zone économique européenne a été signé en août 1992.”

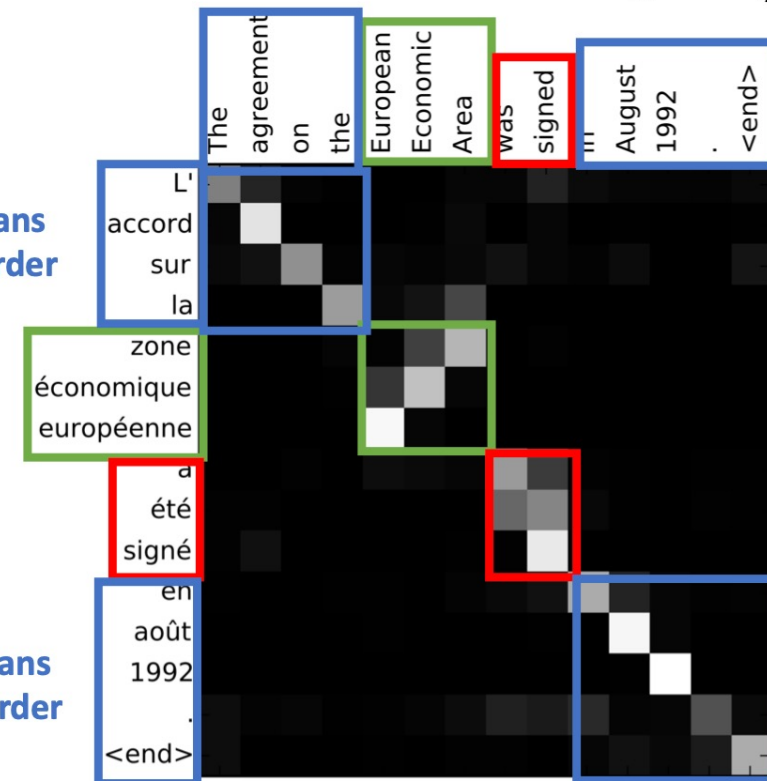
Diagonal attention means words correspond in order

Attention figures out different word orders

Verb conjugation

Diagonal attention means words correspond in order

Visualize attention weights $a_{t,i}$



Transformers

Transformers



Write With Transformer `distil-gpt2` ⓘ

 Shuffle initial text

 Trigger autocomplete or `tab`

Select suggestion `↑` `↓` and `enter`

Cancel suggestion `esc`

I am in Singapore so my favorite food is

curry.

the kohlr

chicken curry, but

<https://transformer.huggingface.co/doc/distil-gpt2>

Transformers

Notable for its use of **attention** to model long-range dependencies in data

A sequence-to-sequence model

Model of choice in natural language processing (NLP)



Step-by-step guide to self-attention with illustrations and code <https://jalammar.github.io/illustrated-transformer/>

Ashish Vaswani et al., [Attention Is All You Need](#), NIPS 2017 (from Google)

Transformers

Like LSTM, Transformer is an architecture for transforming one sequence into another one with the help of two parts (Encoder and Decoder)

But it differs from the existing sequence-to-sequence models because it does not imply any Recurrent Networks (GRU, LSTM, etc.).

- Layer outputs can be calculated in parallel, instead of a series like an RNN
- Attention-based models allow modeling of dependencies without regard to their distance in the input or output sequences

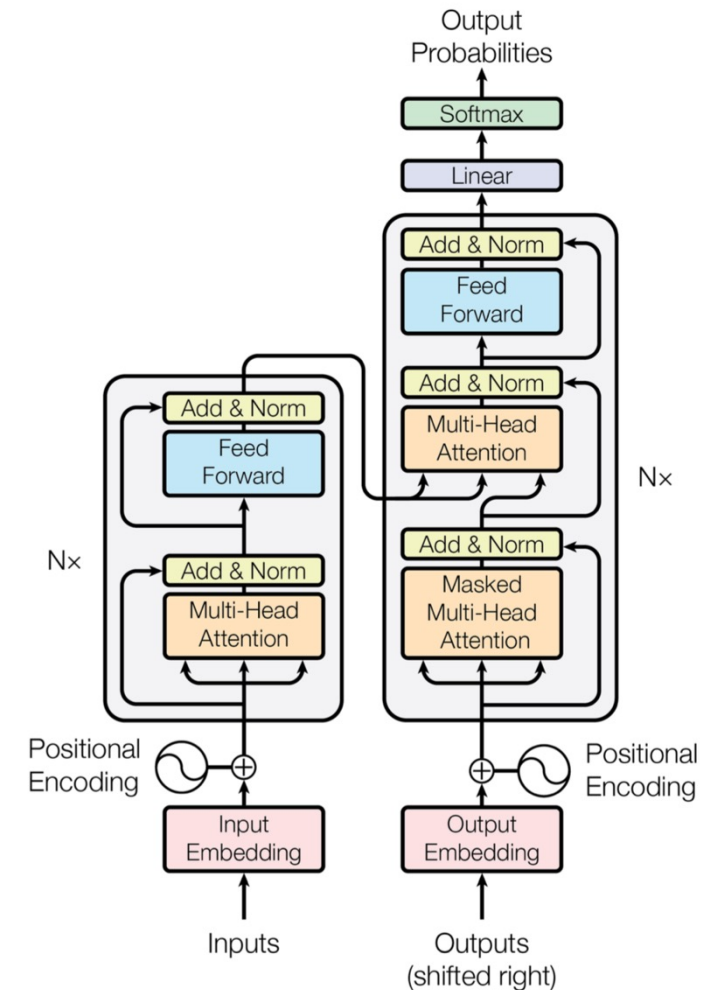
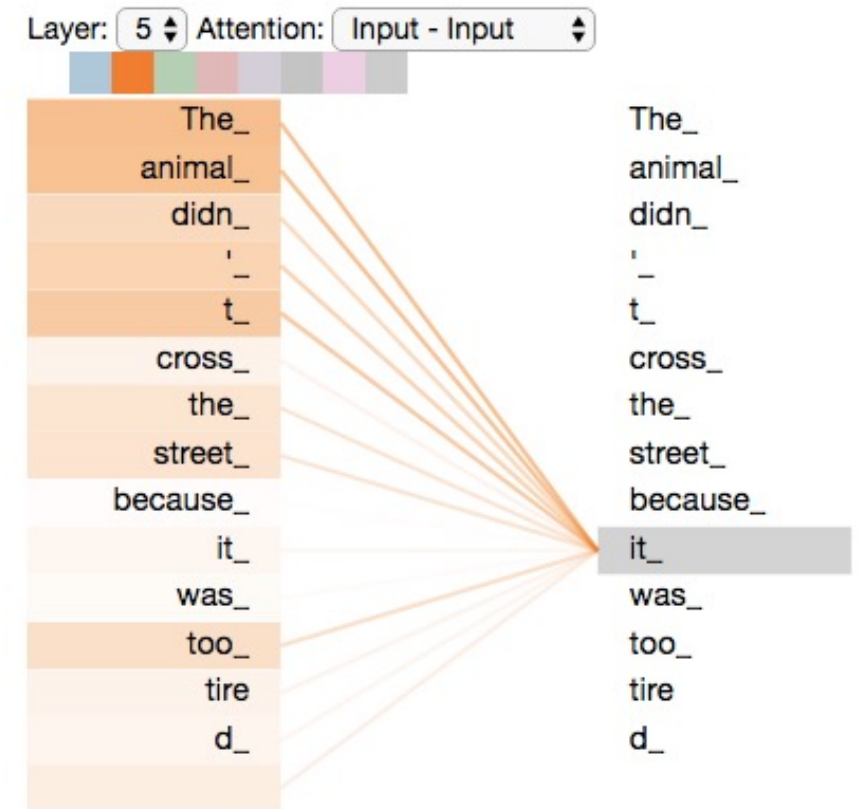


Figure 1: The Transformer - model architecture.

Transformers

It is entirely built on the **self-attention** mechanisms without using sequence-aligned recurrent architecture

Self-attention is a type of attention mechanism where the model makes prediction for one part of a data sample using other parts of the observation about the same sample.



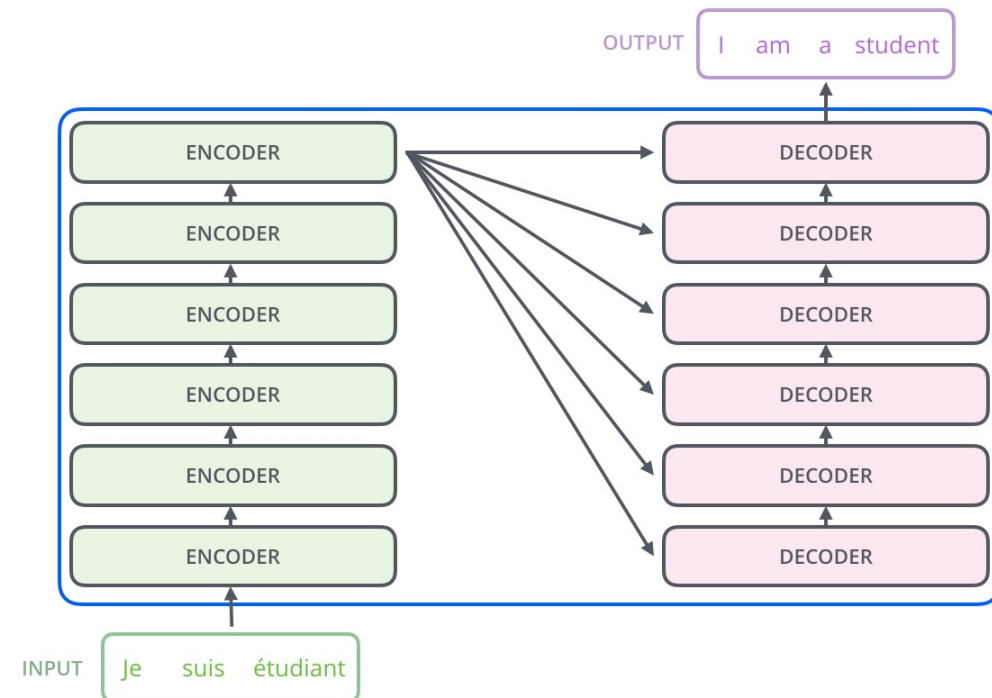
What does “it” in this sentence refer to? Is it referring to the street or to the animal?

When the model is processing the word “it”, self-attention allows it to associate “it” with “animal”.

Transformers

The encoding component is a stack of encoders

The decoding component is a stack of decoders of the same number

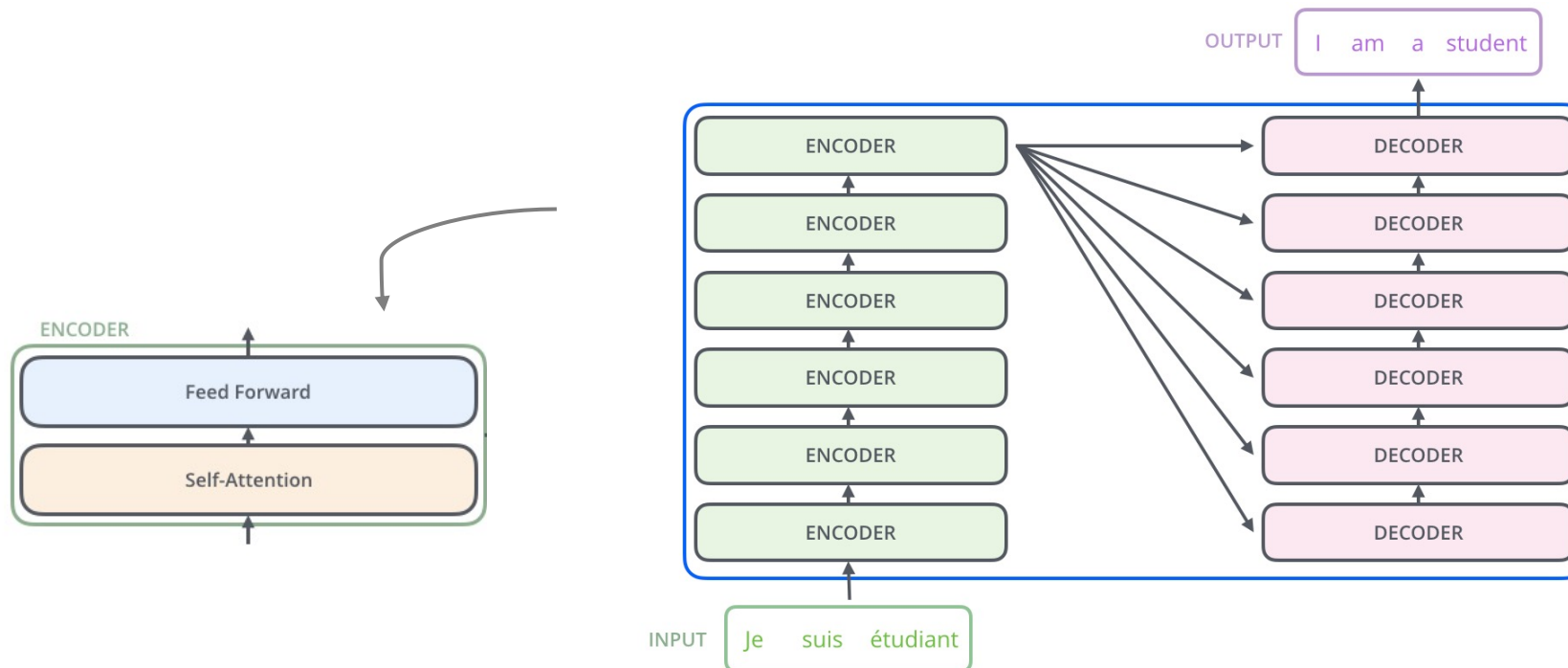


Transformers

The encoder's inputs first flow through a self-attention layer – a layer that **helps the encoder look at other words in the input sentence** as it encodes a specific word

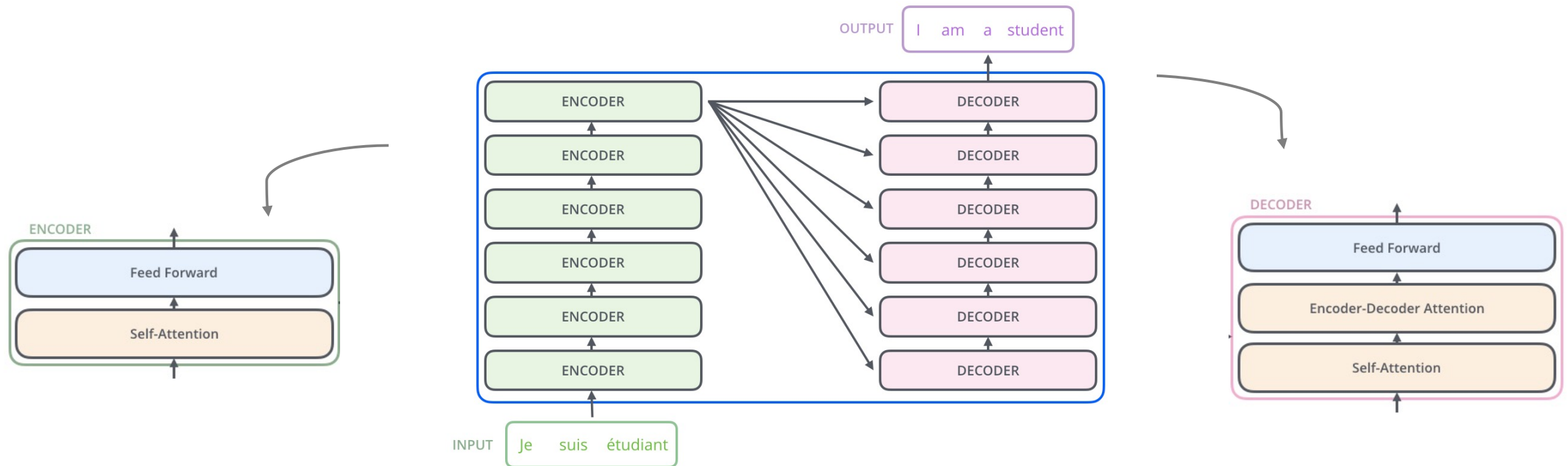
The outputs of the self-attention layer are fed to a **feed-forward neural network**.

The exact same feed-forward network is **independently applied** to each position (each word/token).

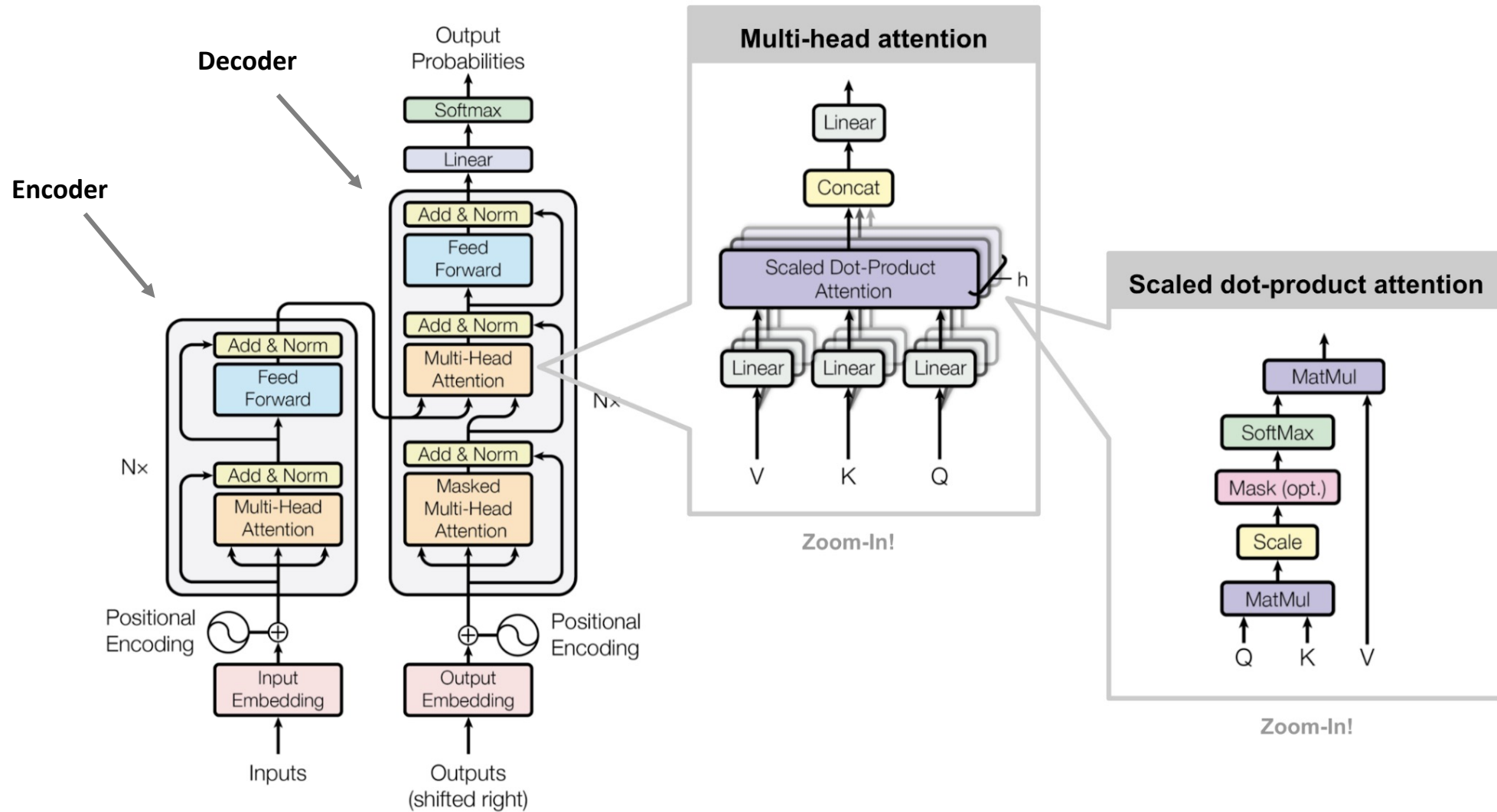


Transformers

The decoder has both those layers, but between them is an attention layer that **helps the decoder focus on relevant parts of the input sentence**



Transformers

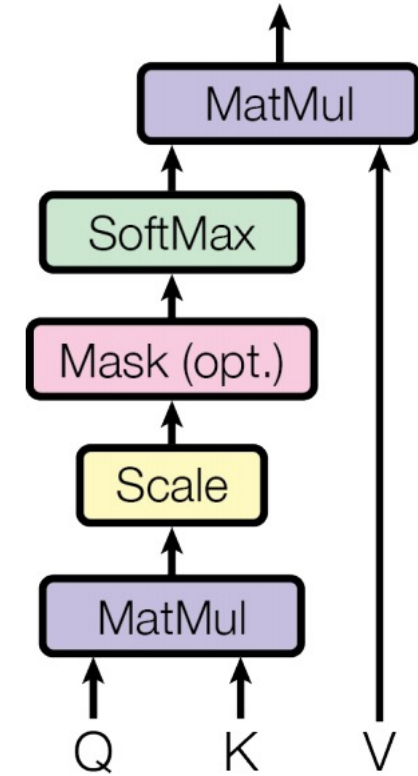


Transformers

Self-attention = Scaled dot-product attention

The output is a weighted sum of the **values**, where the weight assigned to each value is determined by the dot-product of the **query** with all the **keys**

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SoftMax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$



Scaled Dot-Product Attention

Self-Attention in Detail

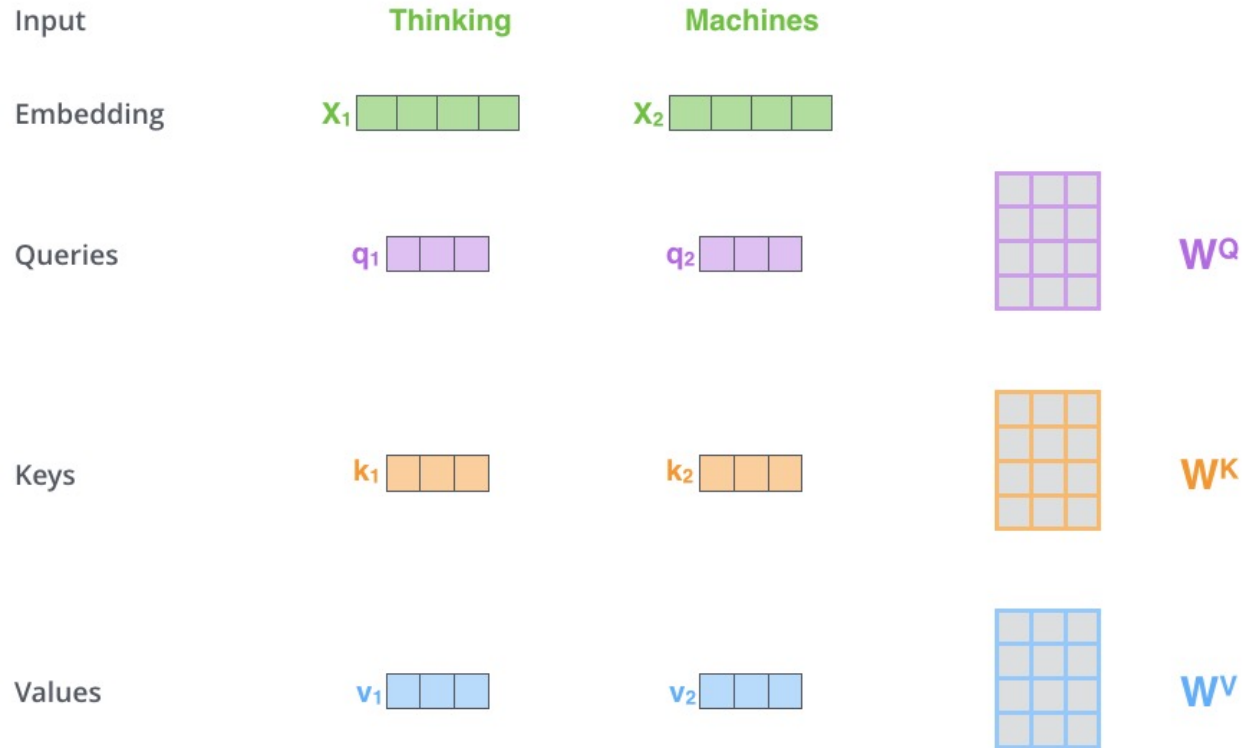
First Step

Create three vectors from each of the encoder's input vectors (in this case, the **embedding** of each word).

So for each word, we create a **Query vector**, a **Key vector**, and a **Value vector**.

These vectors are created by multiplying the embedding by three matrices that we trained during the training process.

What are the “query”, “key”, and “value” vectors?



$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SoftMax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

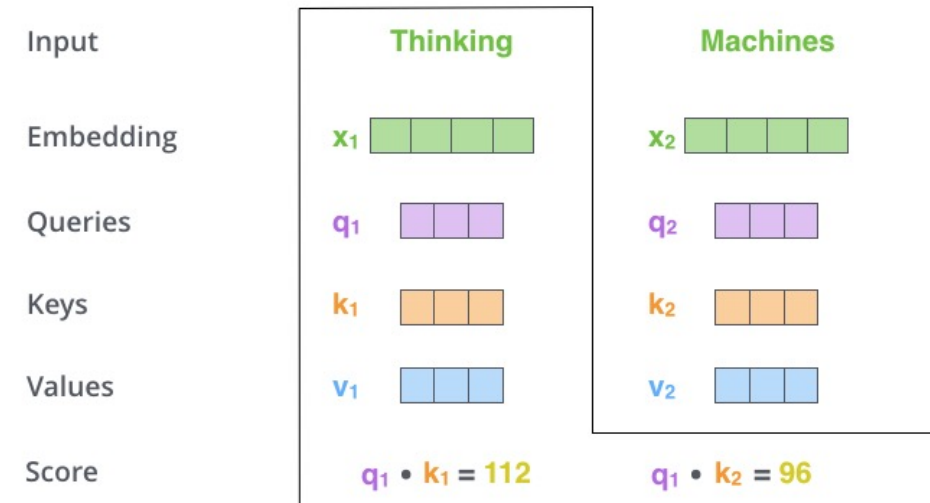
Self-Attention in Detail

Second Step

Calculate a score for each word of the input sentence against a word.

The score determines how much focus to place on other parts of the input sentence as we encode a word at a certain position.

The score is calculated by taking the dot product of the **query vector** with the **key vector** of the respective word we're scoring.



$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SoftMax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

Self-Attention in Detail

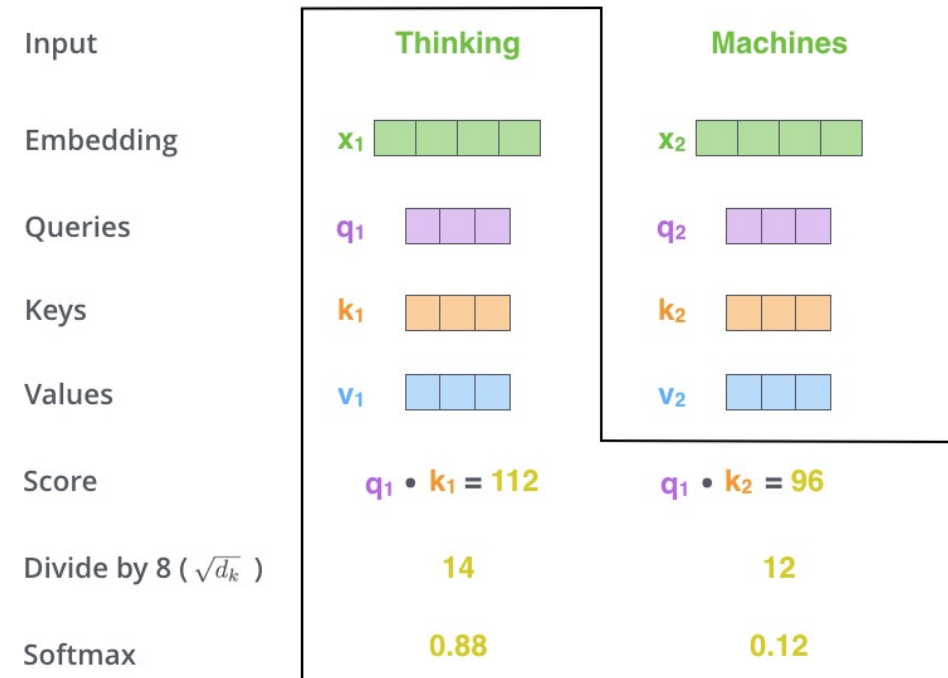
Third Step

Divide the scores by $\sqrt{d_k}$, the square root of the dimension of the key vectors

This leads to having more stable gradients (large similarities will cause softmax to saturate and give vanishing gradients)

Fourth Step

Softmax for normalization



$$z_1 = q_1 \cdot k_1 / \sqrt{d_k} = 112 / \sqrt{64} = 112 / 8 = 14, z_2 = q_2 \cdot k_2 / \sqrt{d_k} = 96 / \sqrt{64} = 96 / 8 = 12$$
$$\text{softmax}(z_1) = \exp(z_1) / \sum_{i=1}^2 (\exp(z_i)) = 0.88, \text{softmax}(z_2) = \exp(z_2) / \sum_{i=1}^2 (\exp(z_i)) = 0.12$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SoftMax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

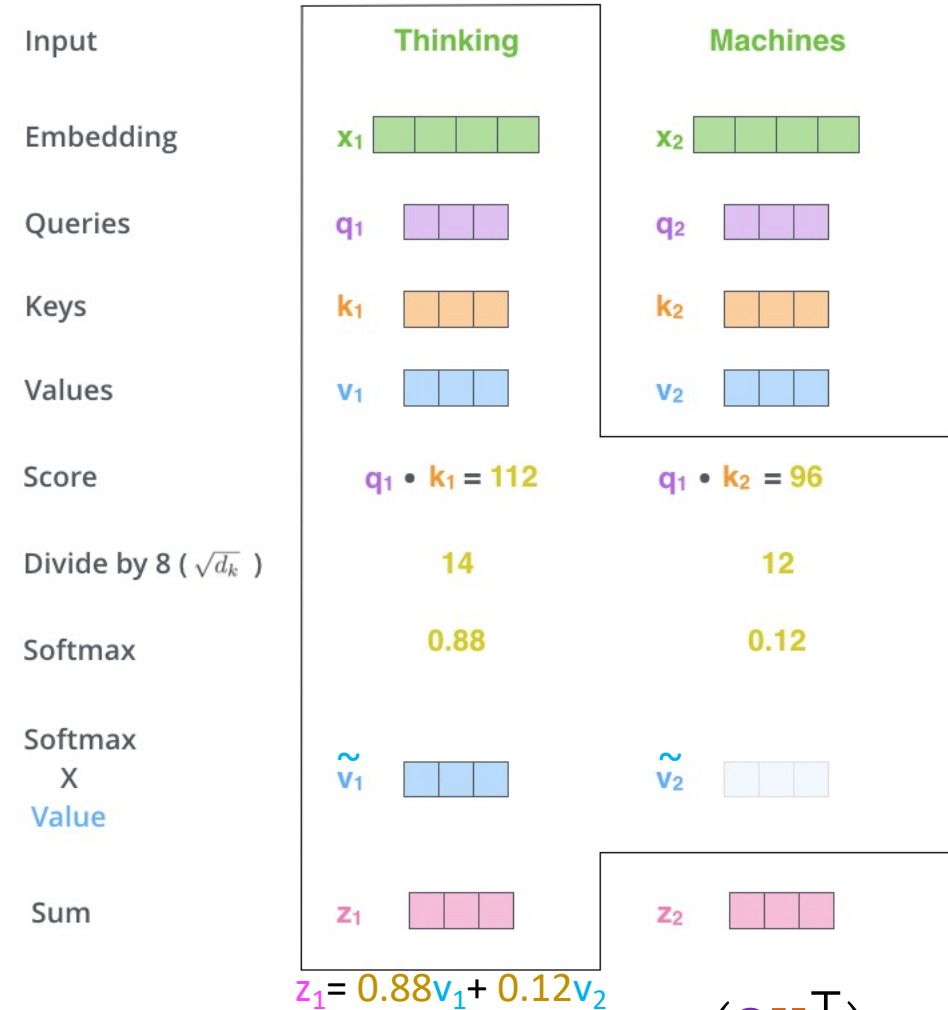
Self-Attention in Detail

Fifth Step

Multiply each value vector by the softmax score

Sixth Step

Sum up the weighted value vectors to get the output of the self-attention layer at this position (for the first word)



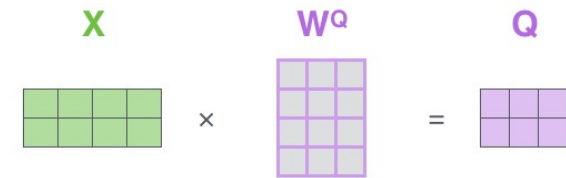
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SoftMax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

Matrix Calculation of Self-Attention

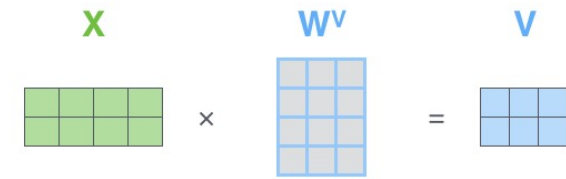
First Step

Calculate the Query, Key, and Value matrices.

Pack our embeddings into a matrix X , and multiplying it by the weight matrices we've trained (W^Q , W^K , W^V)

$$X \times W^Q = Q$$


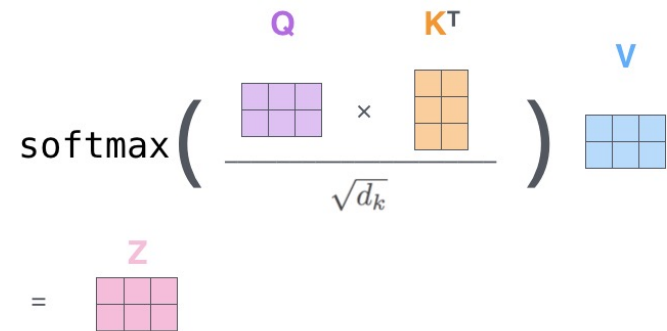
$$X \times W^K = K$$


$$X \times W^V = V$$


Every row in the X matrix corresponds to a word in the input sentence.

Second Step

Calculate the outputs of the self-attention layer.
SoftMax is row-wise

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V = Z$$


Multi-Head Self-Attention

Multi-Head Self-Attention

Rather than only computing the attention once, the multi-head mechanism runs through the scaled dot-product attention **multiple times in parallel**.

The independent attention outputs are simply **concatenated and linearly transformed** into the expected dimensions.

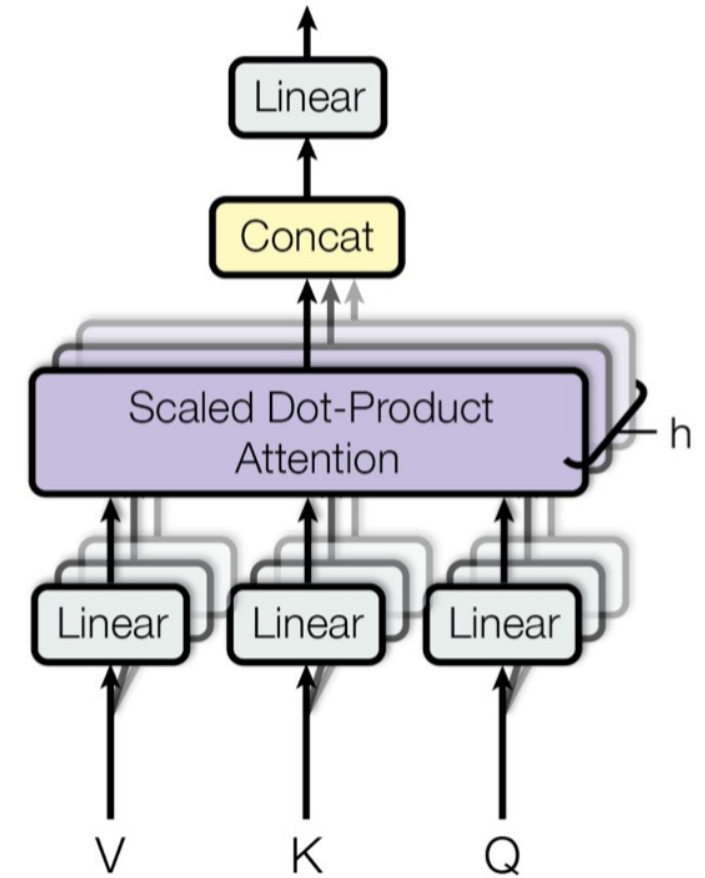
Why?

*“Multi-head attention allows the model to jointly attend to information from different representation **subspaces** at different positions.”*

Example:

Deep learning (also known as deep structured learning) is part of a broader family of **machine learning methods** based on artificial neural networks with **representation learning**.

Given “**representation learning**”, the first head attends to “**Deep learning**” while the second head attends to the more general term “**machine learning methods**”



Multi-head Attention

Positional Encoding

Self-attention operation is permutation equivariant

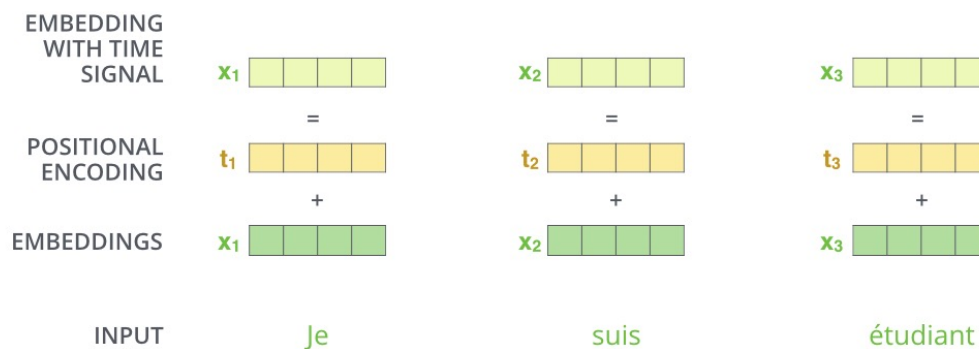
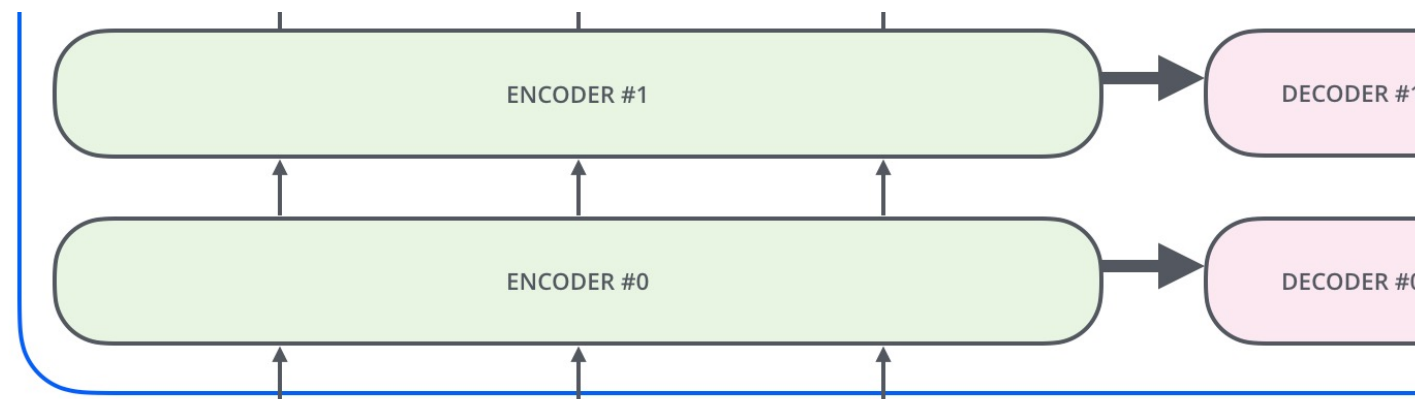
$$\text{SelfAtt}(\pi \cdot X) = \pi \cdot \text{SelfAtt}(X)$$

Self-attention layer works on sets of vectors and it doesn't know the order of the vectors it is processing

The positional encoding has the same dimension as the input embedding

Adds a vector to each input embedding to give information about the relative or absolute position of the tokens in the sequence

These vectors follow a specific pattern



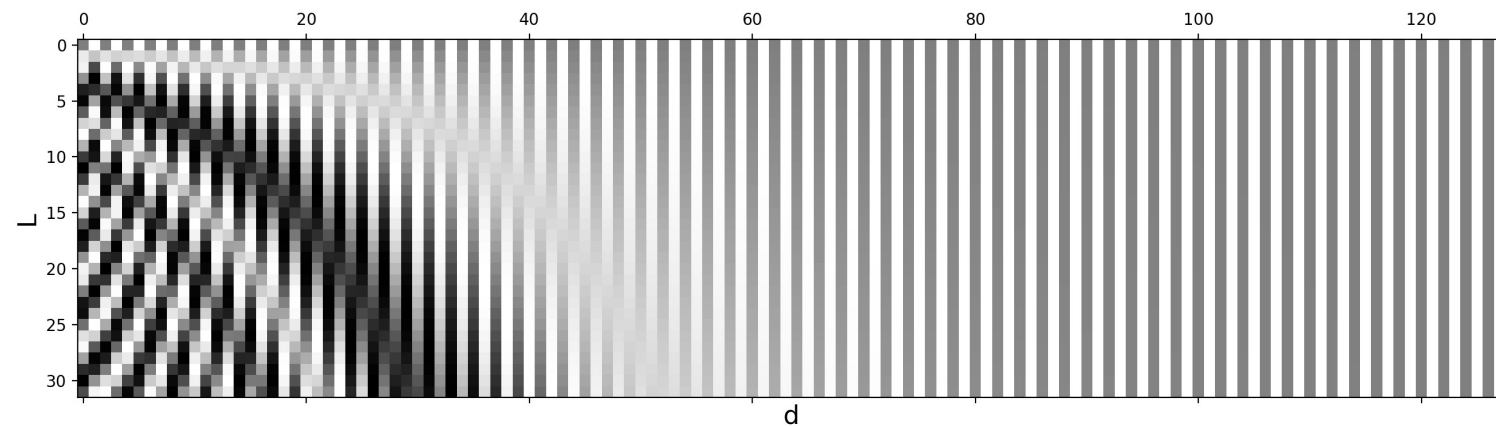
Positional Encoding

What might this pattern look like?

Each row corresponds to a positional encoding of a vector.

The first row would be the vector we'd add to the embedding of the first word in an input sequence.

Each position is uniquely encoded and the encoding can deal with sequences longer than any sequence seen in the training time.



Sinusoidal positional encoding with 32 tokens and embedding dimension of 128. The value is between -1 (black) and 1 (white) and the value 0 is in gray.

Sinusoidal positional encoding - interweaves the two signals

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension, $[0, \dots, d_{\text{model}}/2)$

Example: Positional Encoding

Example:

Given the following *Sinusoidal positional encoding*, calculate the $PE(pos = 1)$ for the first five dimensions $[0, 1, 2, 3, 4]$. Assume $d_{model} = 512$

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Solution:

Given $pos = 1$ and $d_{model} = 512$

At dimension 0, $2i = 0$ thus $i = 0$, therefore $PE_{(pos, 2i)} = PE_{(1, 0)} = \sin(1/10000^{0/512})$

At dimension 1, $2i + 1 = 1$ thus $i = 0$, therefore $PE_{(pos, 2i+1)} = PE_{(1, 1)} = \cos(1/10000^{0/512})$

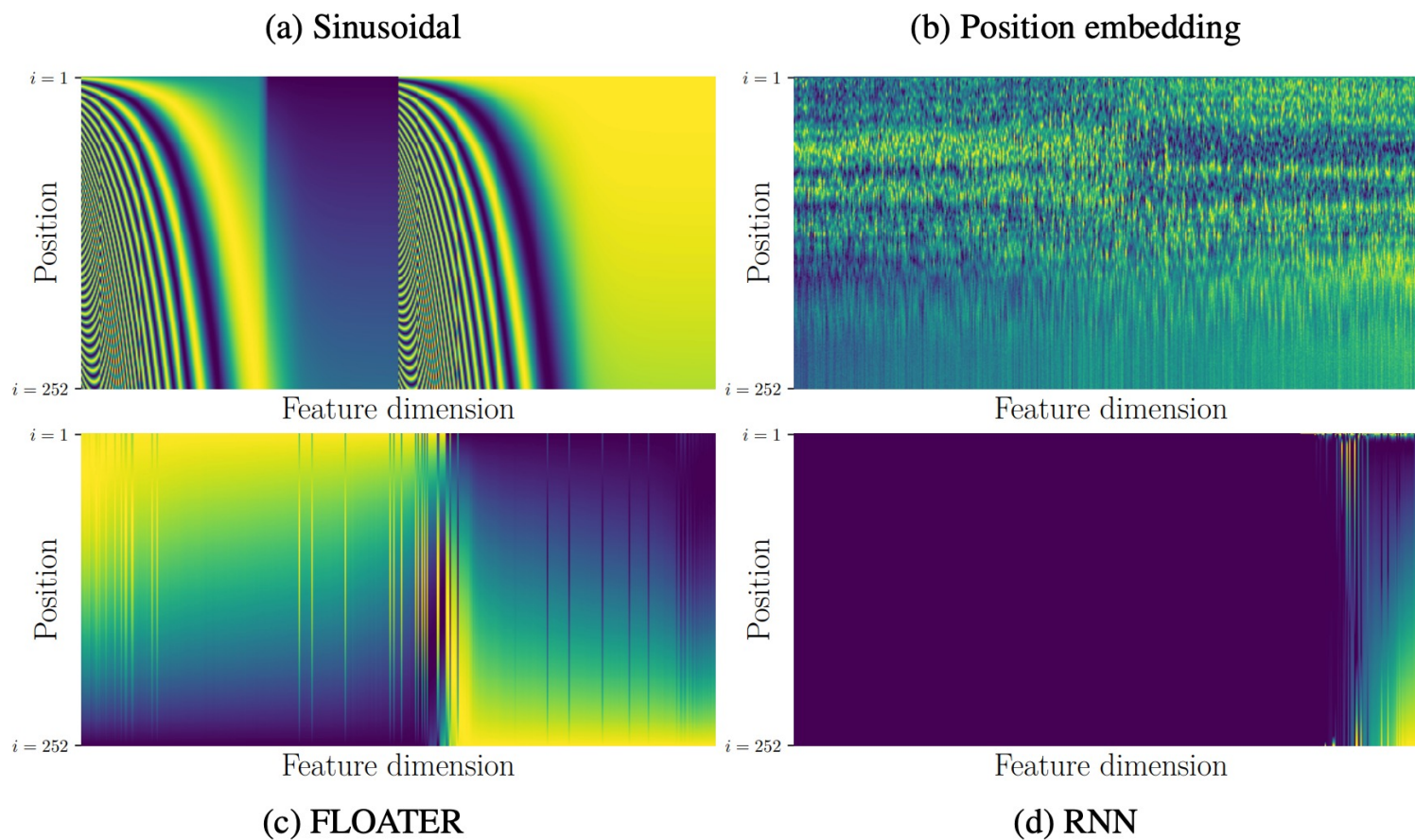
At dimension 2, $2i = 2$ thus $i = 1$, therefore $PE_{(pos, 2i)} = PE_{(1, 2)} = \sin(1/10000^{2/512})$

At dimension 3, $2i + 1 = 3$ thus $i = 1$, therefore $PE_{(pos, 2i+1)} = PE_{(1, 3)} = \cos(1/10000^{2/512})$

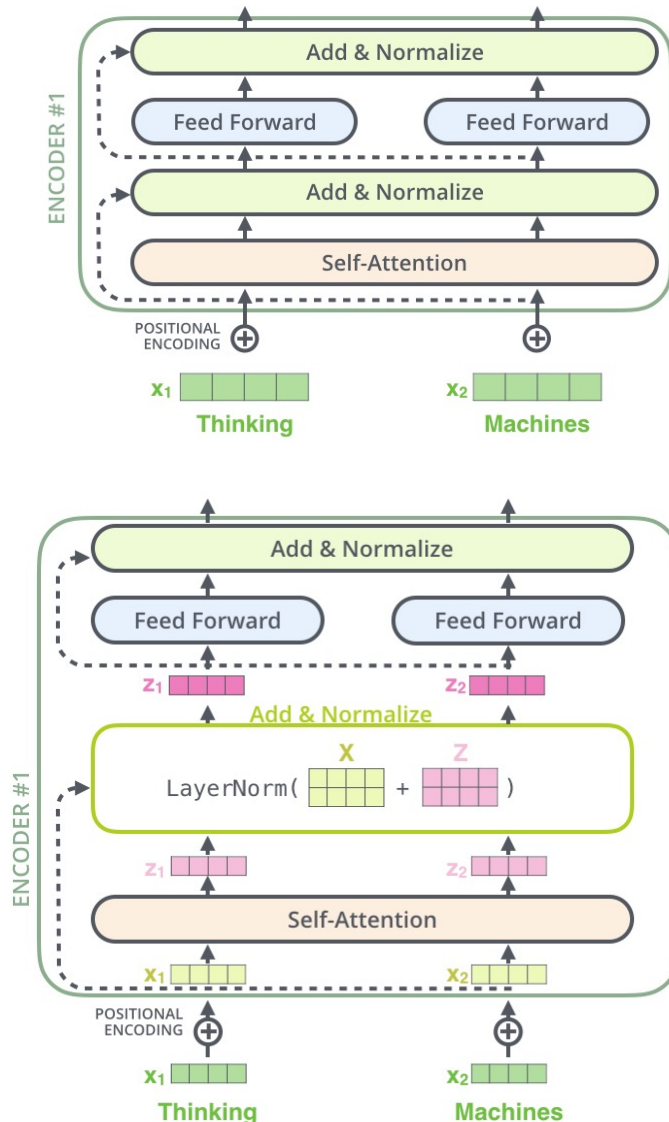
At dimension 4, $2i = 4$ thus $i = 2$, therefore $PE_{(pos, 2i)} = PE_{(1, 4)} = \sin(1/10000^{4/512})$

Positional Encoding

Other form of positional encoding (e.g., learnable encoding)



Transformer Encoder



If we're to visualize the vectors and the layer-norm operation associated with self attention, it would look like this:

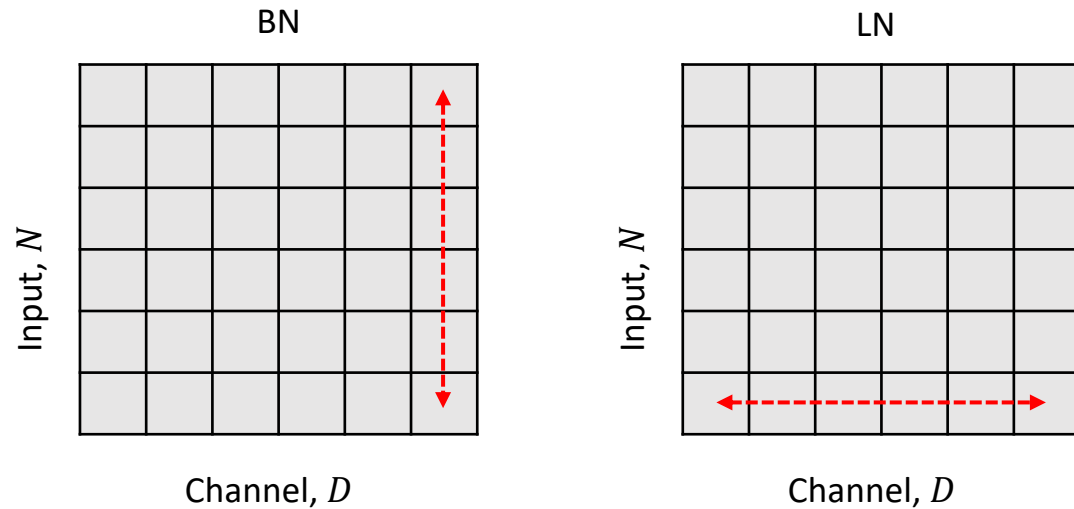
Encoder

- A stack of $N = 6$ identical layers.
- Each layer has a multi-head self-attention layer and a simple position-wise fully connected feed-forward network.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

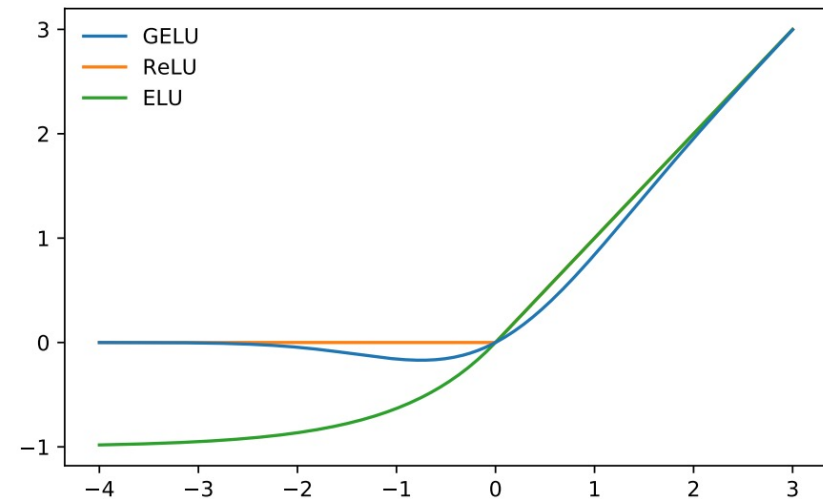
- The linear transformations are the same across different positions, they use different parameters from layer to layer.
- Each sub-layer adopts a residual connection and a layer normalization.

Transformer Encoder



Layer Normalization (LN) ¹

- The pixels along the red arrow are normalized by the same mean and variance, computed by aggregating the values of these pixels.
- BN is found unstable in Transformers ²
- Works well with RNN and now being used in Transformers



Gaussian Error Linear Units (GELU) ³

- Can be thought of as a smoother ReLU
- Used in GPT-3, BERT, and most other Transformers

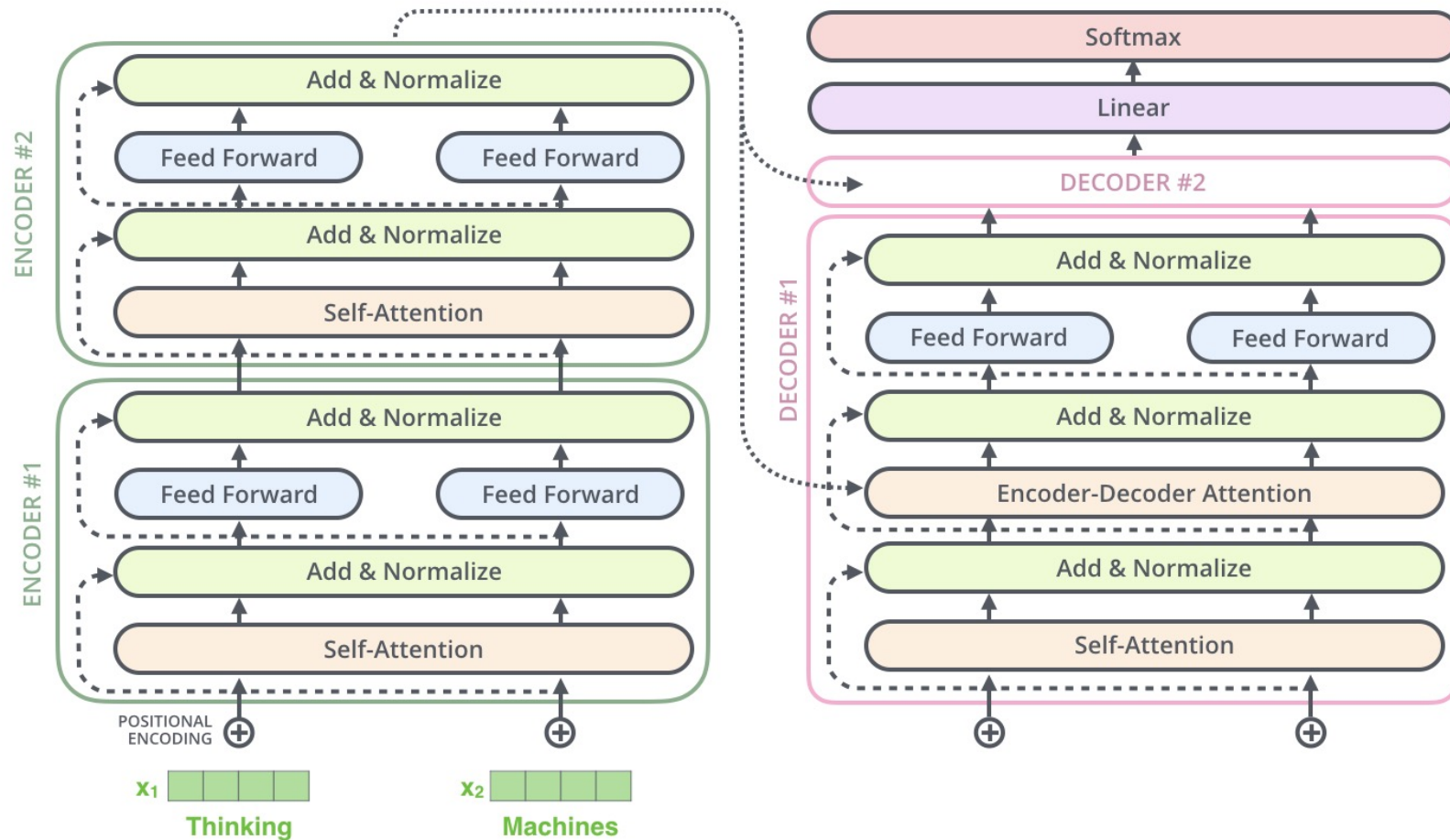
¹ Jimmy Lei Ba, Jamie Ryan Kiros, Geoffrey E. Hinton, [Layer Normalization](#), arXiv:1607.06450

² Sheng Shen, Zhewei Yao, Amir Gholami, Michael Mahoney, Kurt Keutzer, [Rethinking Batch Normalization in Transformers](#), ICML 2020

³ Dan Hendrycks, Kevin Gimpel, [Gaussian Error Linear Units \(GELUs\)](#), arXiv:1606.08415

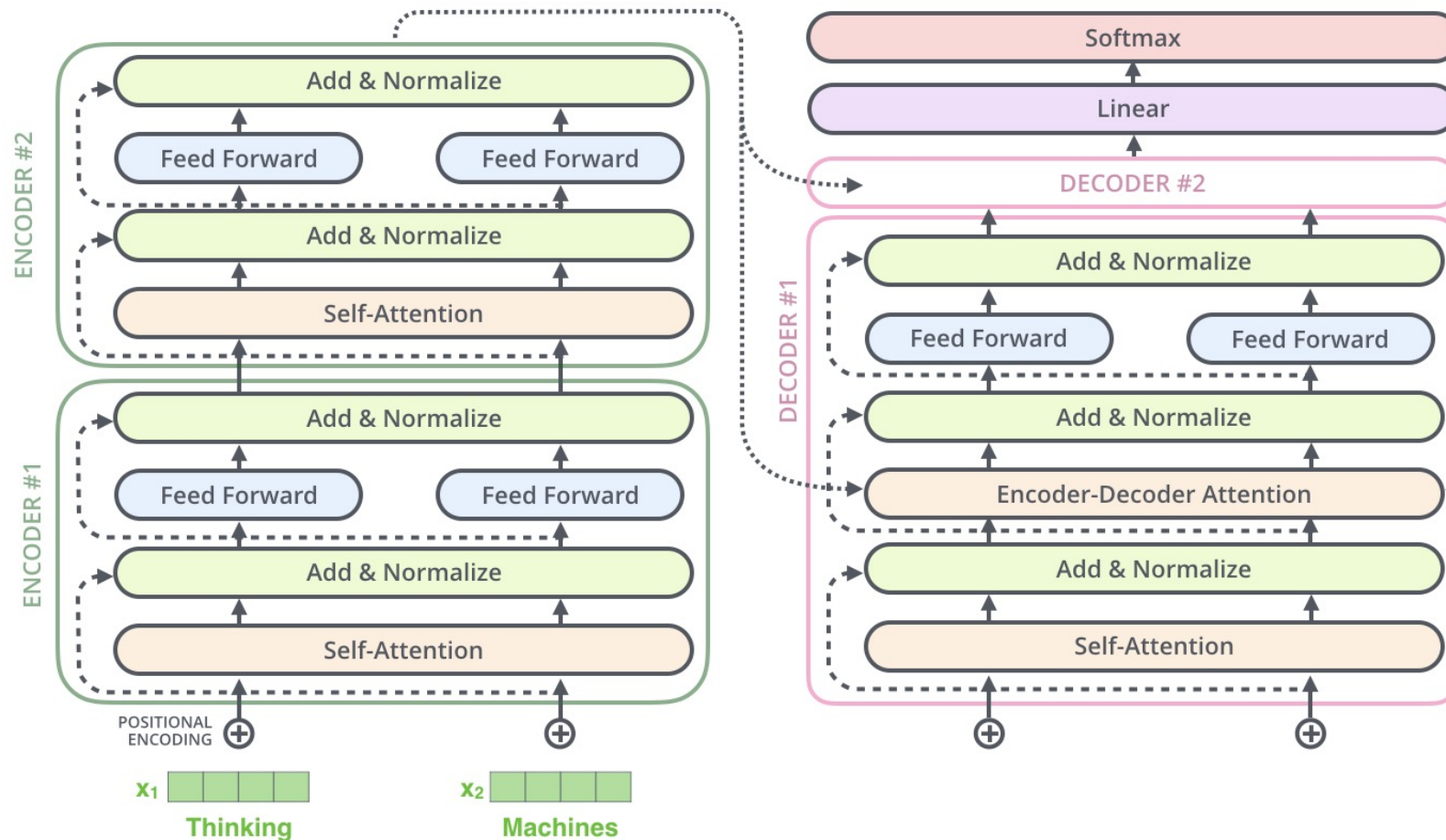
Transformer Encoder

The outputs of encoder go to the sub-layers of the decoder as well. If we're to think of a Transformer of two stacked encoders and decoders, it would look something like this:



Transformer Encoder

The outputs of encoder go to the sub-layers of the decoder as well. If we're to think of a Transformer of two stacked encoders and decoders, it would look something like this:

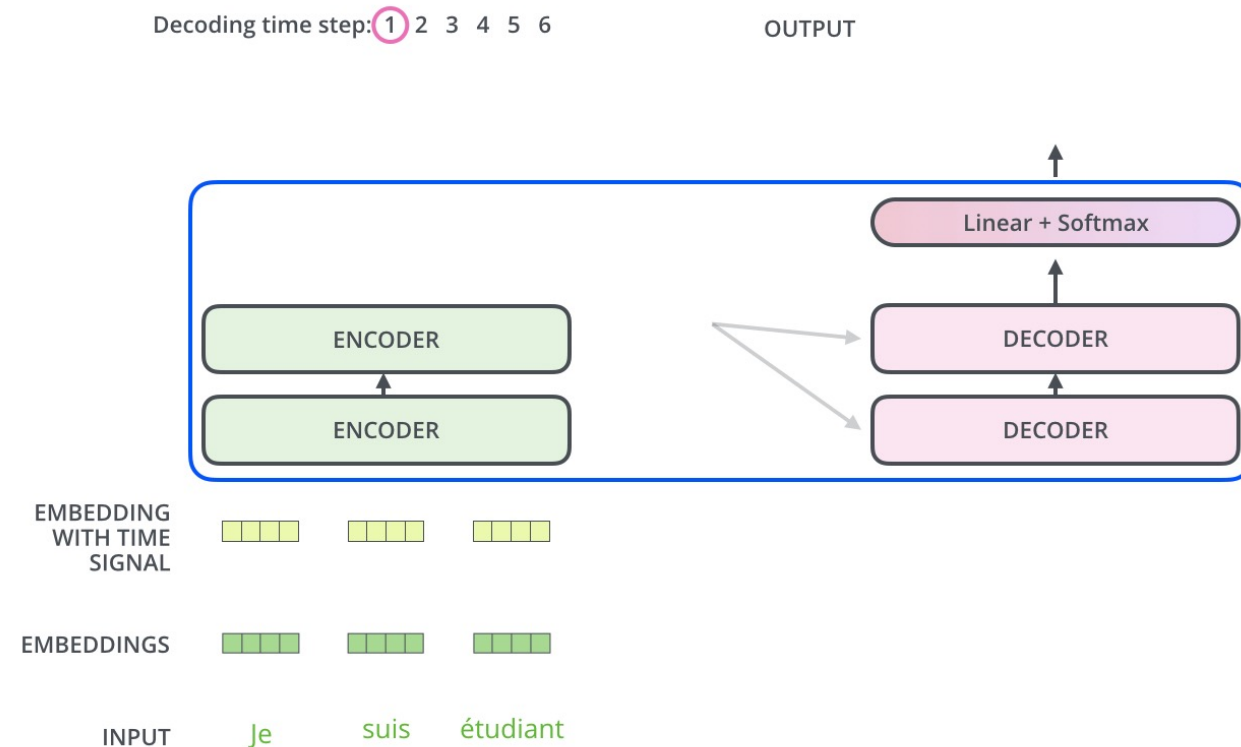


The “Encoder-Decoder Attention” layer works just like multiheaded self-attention, except it creates its Queries matrix from the layer below it, and takes the Keys and Values matrix from the output of the encoder stack.

Transformer Decoder

How encoder and decoder work together

- The encoder start by processing the input sequence
- The output of the top encoder is then transformed into a set of attention vectors **K** and **V**.
- These are to be used by each decoder in its “encoder-decoder attention” layer which helps the decoder focus on appropriate places in the input sequence



After finishing the encoding phase, we begin the decoding phase. Each step in the decoding phase outputs an element from the output sequence (the English translation sentence in this case).

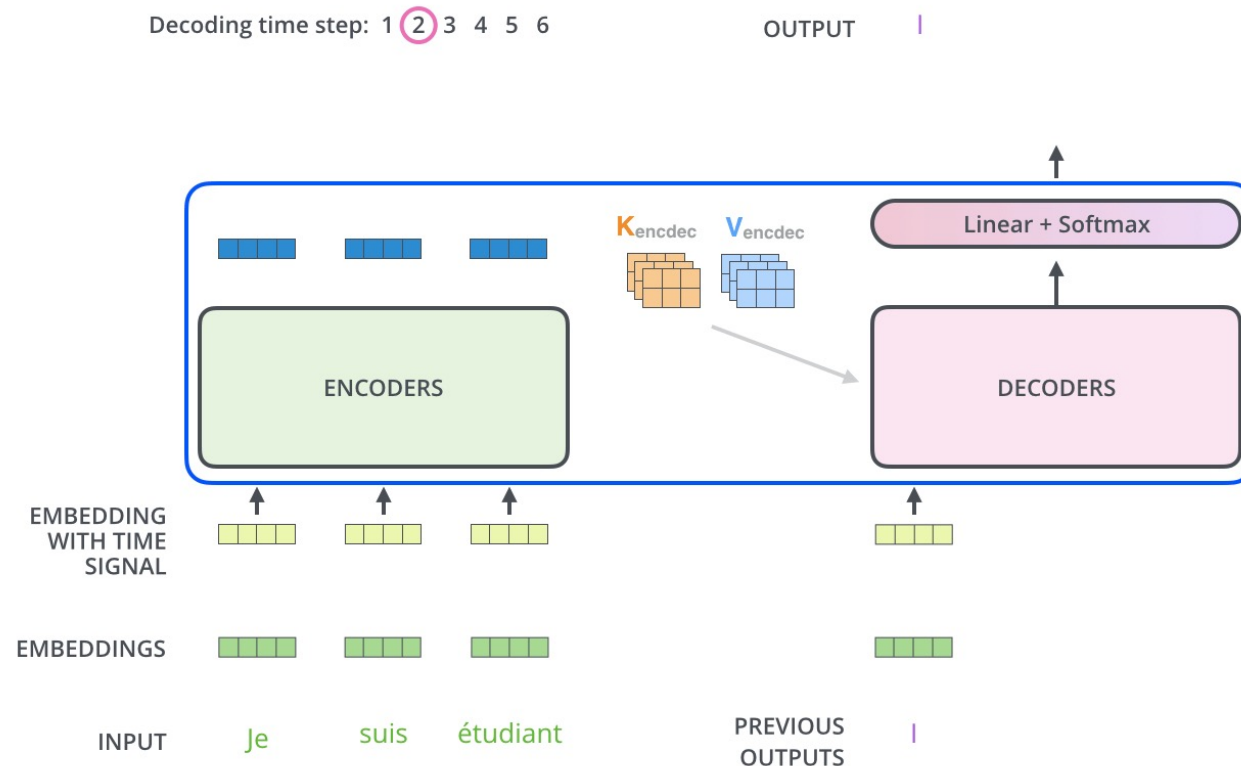
Transformer Decoder

How encoder and decoder work together

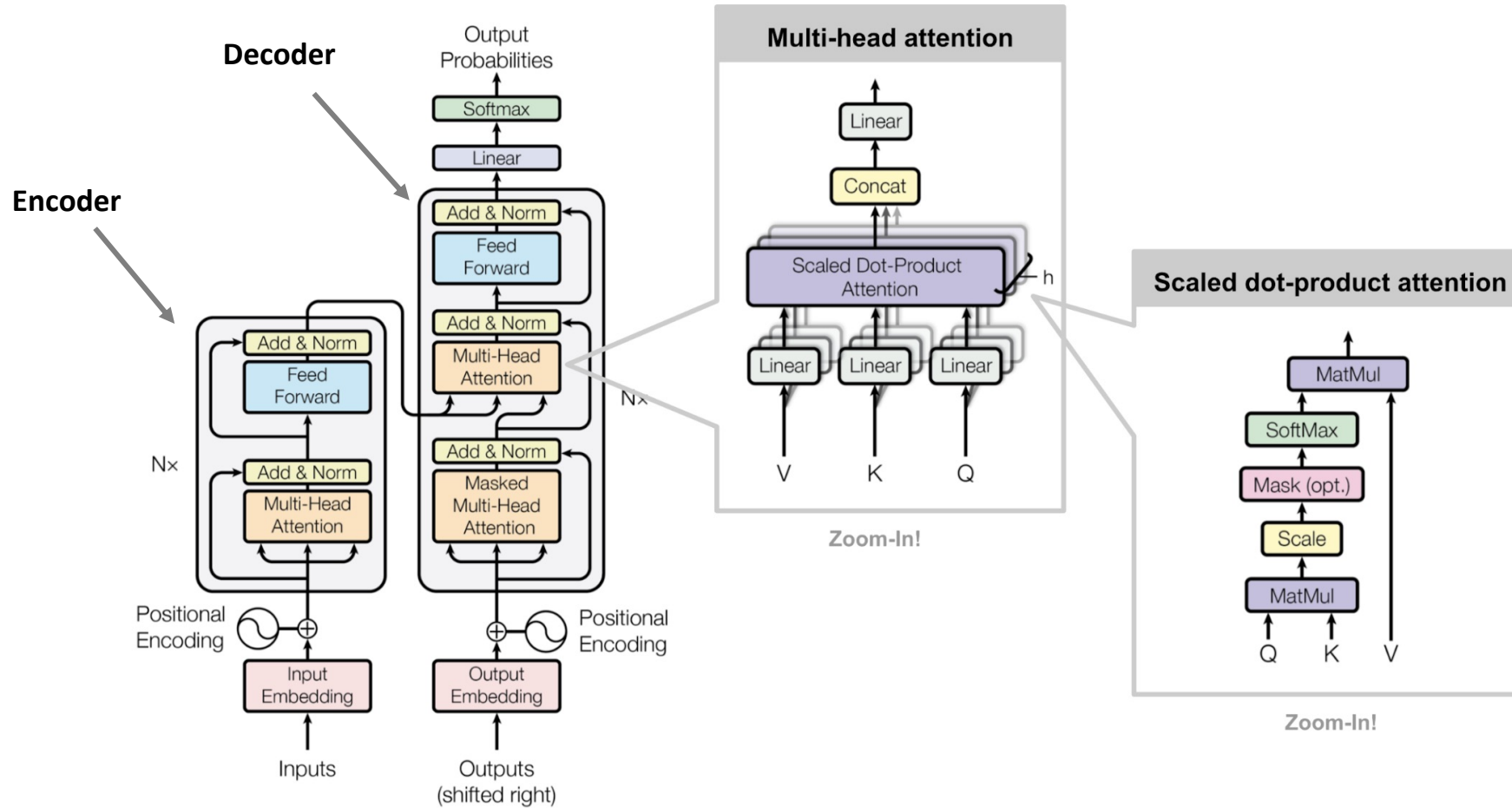
- The output of each step is fed to the bottom decoder in the next time step
- Embed and add positional encoding to those decoder inputs. Process the inputs
- Repeat the process until a special symbol is reached indicating the transformer decoder has completed its output.

Note:

In the decoder, the *self-attention layer* is only allowed to attend to earlier positions in the output sequence. This is done by **masking future positions** (setting them to $-\infty$) before the softmax step in the self-attention calculation.



Transformers



John Thickstun, [The Transformer Model in Equations](#)

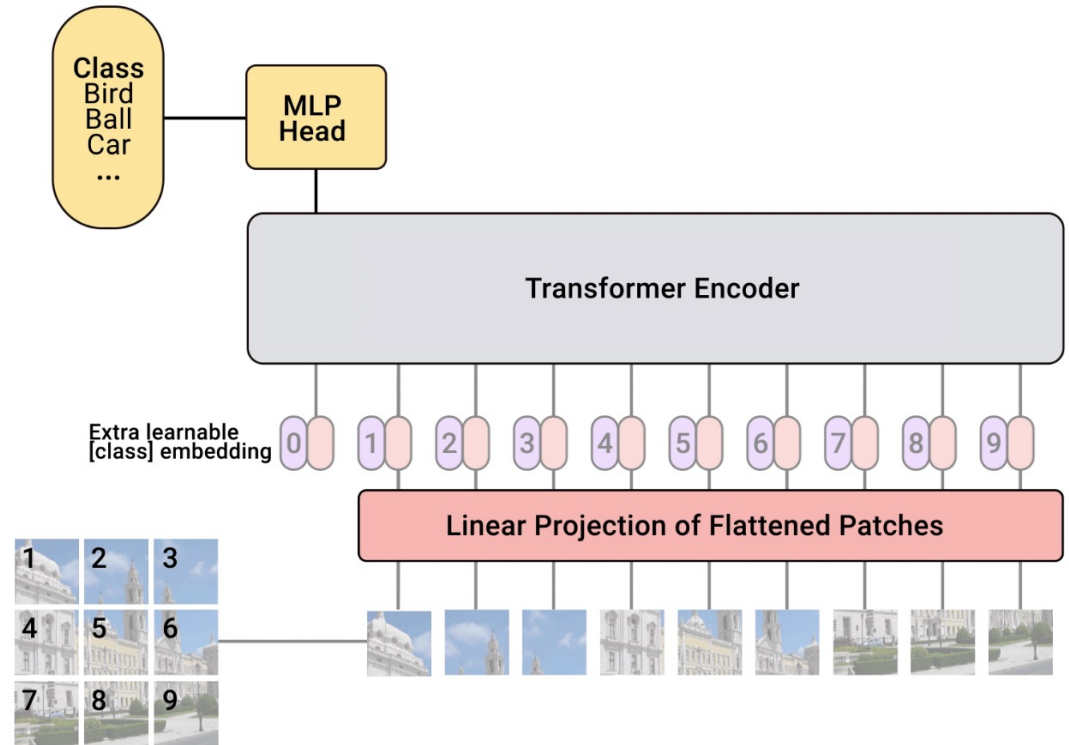
Vision Transformers

Vision Transformer (ViT)

Vision Transformer

- Do not have decoder
- Reshape the image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ into a sequence of flattened 2D patches $\mathbf{x} \in \mathbb{R}^{N \times (P^2 \cdot C)}$, where (H, W) is the resolution of the original image, C is the number of channels, (P, P) is the resolution of each image patch, and $N = HW/P^2$ is the resulting number of patches
- **Patch embedding** - Linearly embed each of them to D dimension with a trainable linear projection $\mathbf{E}$
- Add **learnable** position embeddings $\mathbf{E}_{pos}$ to retain positional information

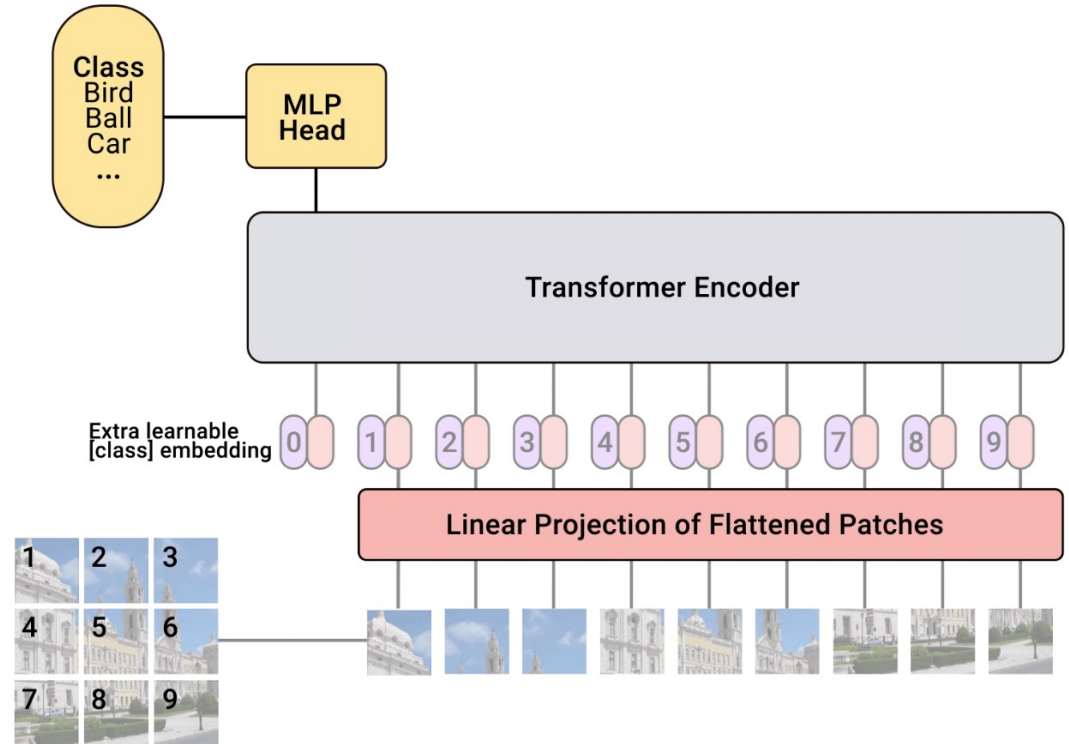
$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \cdots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{pos}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \quad \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$$



Vision Transformer (ViT)

Vision Transformer

- Prepend a learnable embedding ($\mathbf{z}_0^0 = \mathbf{x}_{\text{class}}$) to the sequence of embedded patches
- Feed the resulting sequence of vectors to a standard Transformer encoder
- A classification head is attached to $\mathbf{z}_L^0$



$$\begin{aligned}
 \mathbf{z}_0 &= [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}, & \mathbf{E} &\in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D} \\
 \mathbf{z}'_\ell &= \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, & \ell &= 1 \dots L \\
 \mathbf{z}_\ell &= \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, & \ell &= 1 \dots L \\
 \mathbf{y} &= \text{LN}(\mathbf{z}_L^0)
 \end{aligned}$$

Transformer Encoder {

Classification Head {

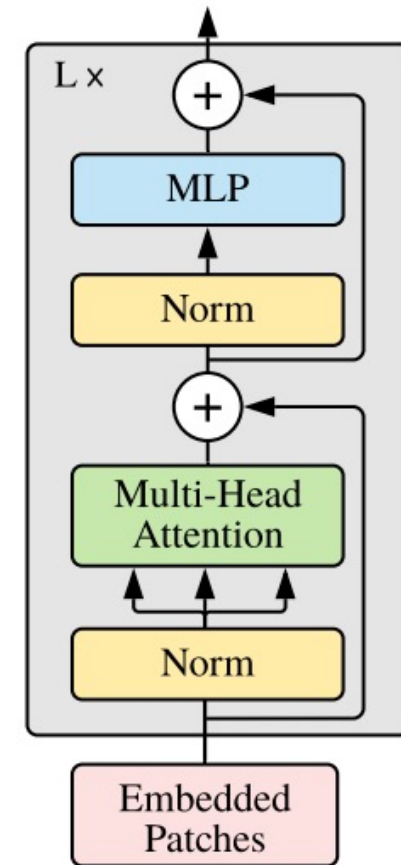
Vision Transformer (ViT)

Transformer Encoder

- Consists of a multi-head self-attention module (**MSA**), followed by a 2-layer MLP (with **GELU**)
- LayerNorm (LN) is applied before MSA module and MLP, and a residual connection is applied after each module.

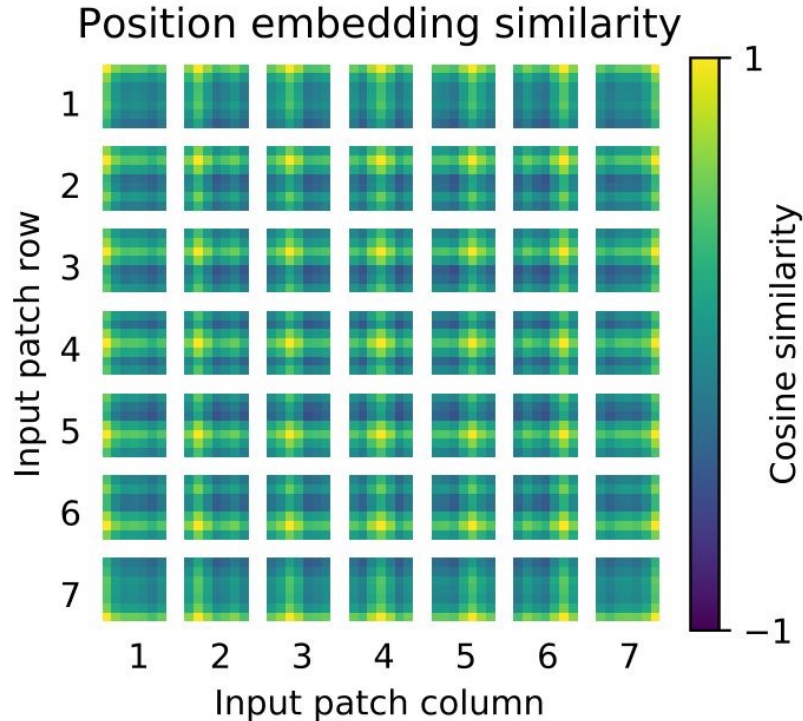
$$\mathbf{z}'_{\ell} = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1},$$

$$\mathbf{z}_{\ell} = \text{MLP}(\text{LN}(\mathbf{z}'_{\ell})) + \mathbf{z}'_{\ell},$$



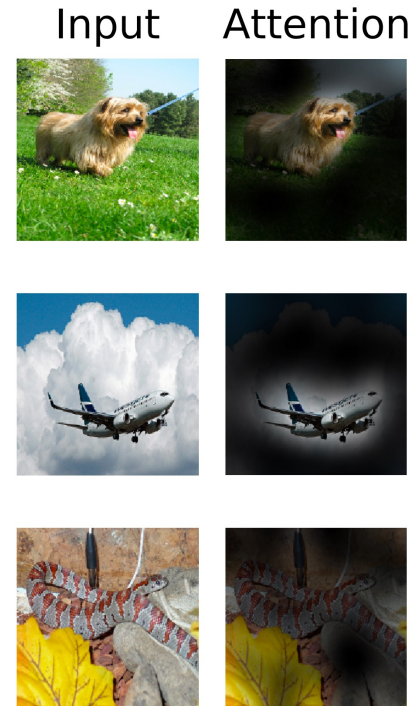
Transformer Encoder

Vision Transformer (ViT)



The model learns to encode distance within the image - closer patches tend to have more similar position embeddings.

The row-column structure appears - patches in the same row/column have similar embeddings.



The model attends to image regions that are semantically relevant for classification

Attention Rollout¹ - averaged attention weights of ViT- L/16 across all heads and then recursively multiplied the weight matrices of all layers

¹ Samira Abnar and Willem Zuidema. [Quantifying attention flow in transformers](#). In ACL, 2020 (allows attention to be more meaningfully visualized and interpreted for deeper layers in a transformer)

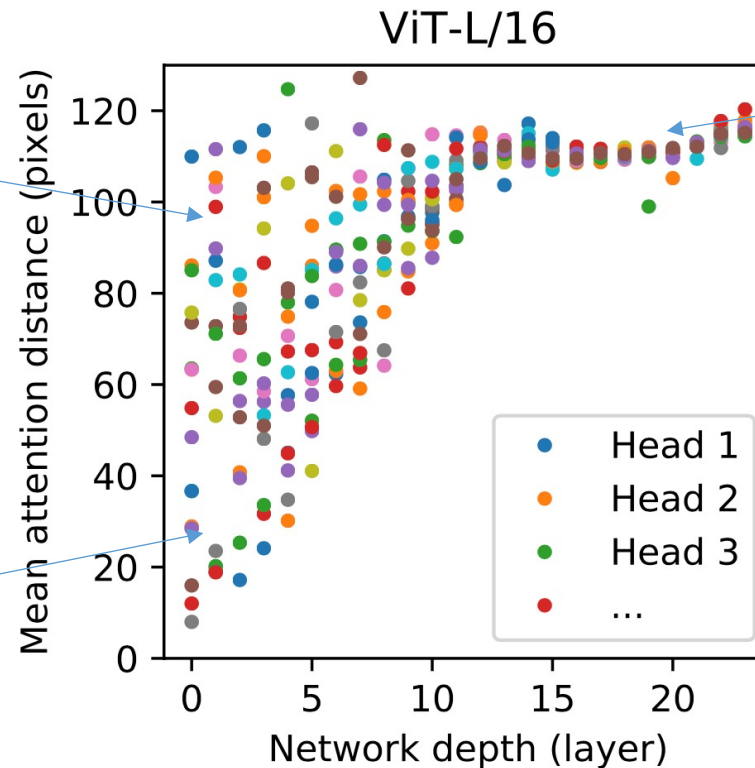
Vision Transformer (ViT)

Examine the attention distance, analogous to receptive field in CNN

Compute the average distance in image space across which information is integrated, based on the attention weights.

Some heads attend to most of the image already in the lowest layers, showing the capability of ViT in integrating information globally

Other attention heads have consistently small attention distances in the low layers

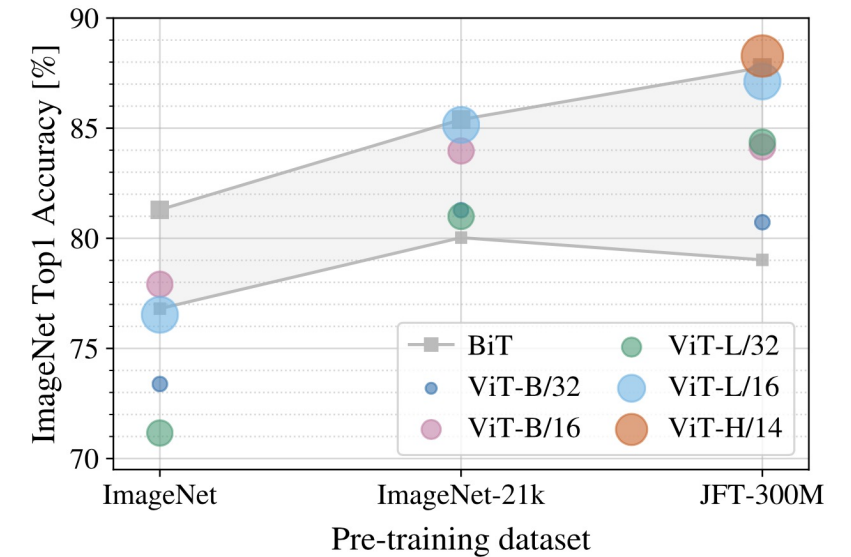


The attention distance increases with network depth

Vision Transformer (ViT)

Performance of ViT

- ViT performs significantly worse than the CNN equivalent (BiT) when trained on ImageNet (1M images).
- However, on ImageNet-21k (14M images) performance is comparable, and on JFT (300M images), ViT outperforms BiT.
- ViT overfits the ImageNet task due to its lack of inbuilt knowledge about images



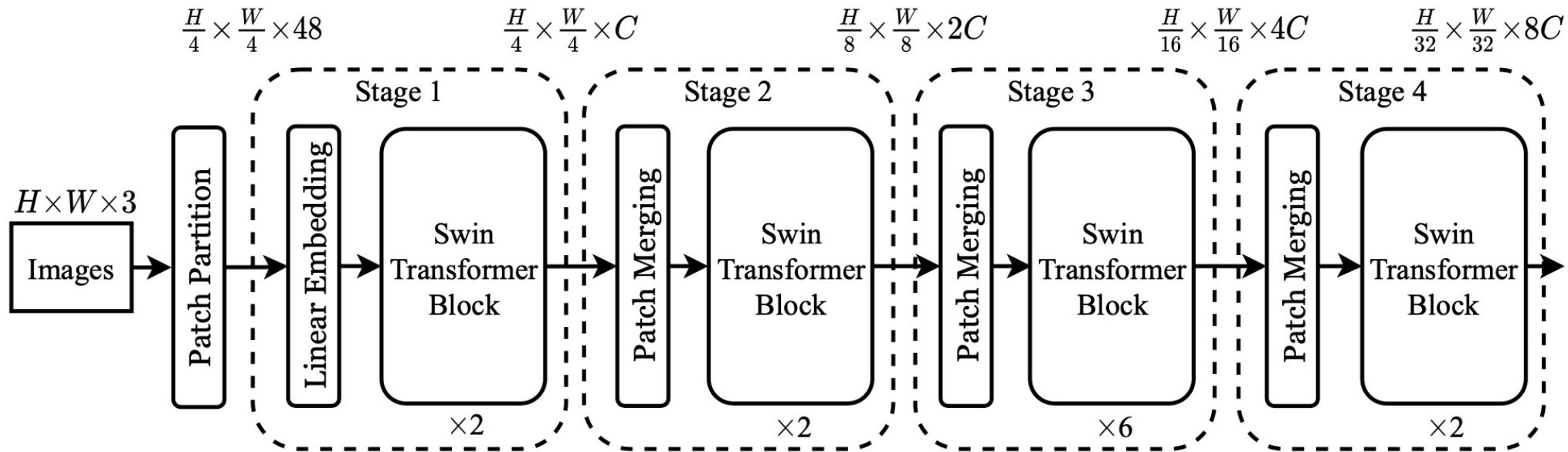
ViT conducts **global self-attention**

- Relationships between a token and all other tokens are computed
- Quadratic complexity with respect to the number of tokens
- Not tractable for dense prediction or to represent a high-resolution image

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Swin Transformers

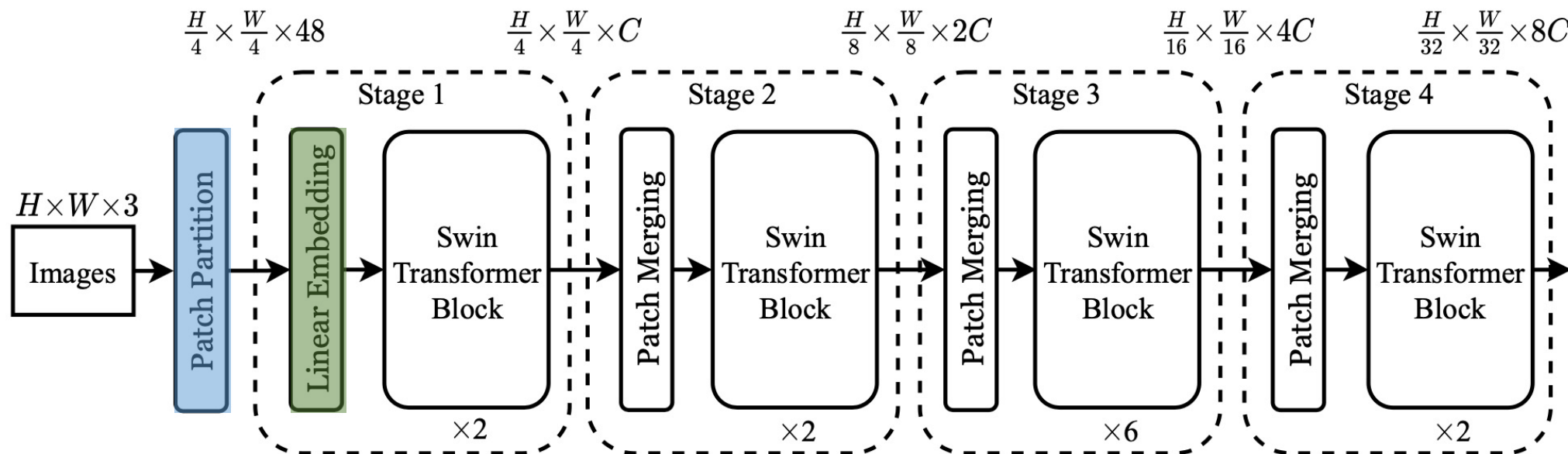
Swin Transformer



Swin Transformer

- Perform local self-attention thus having **linear** computational complexity with respect to the number of tokens
- **Shifted window** between consecutive self-attention layers to allow connections
- Flexibility to model at various scales with **hierarchical** representation

Swin Transformer

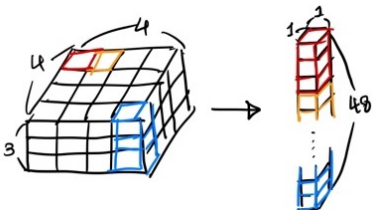


Patch Partition

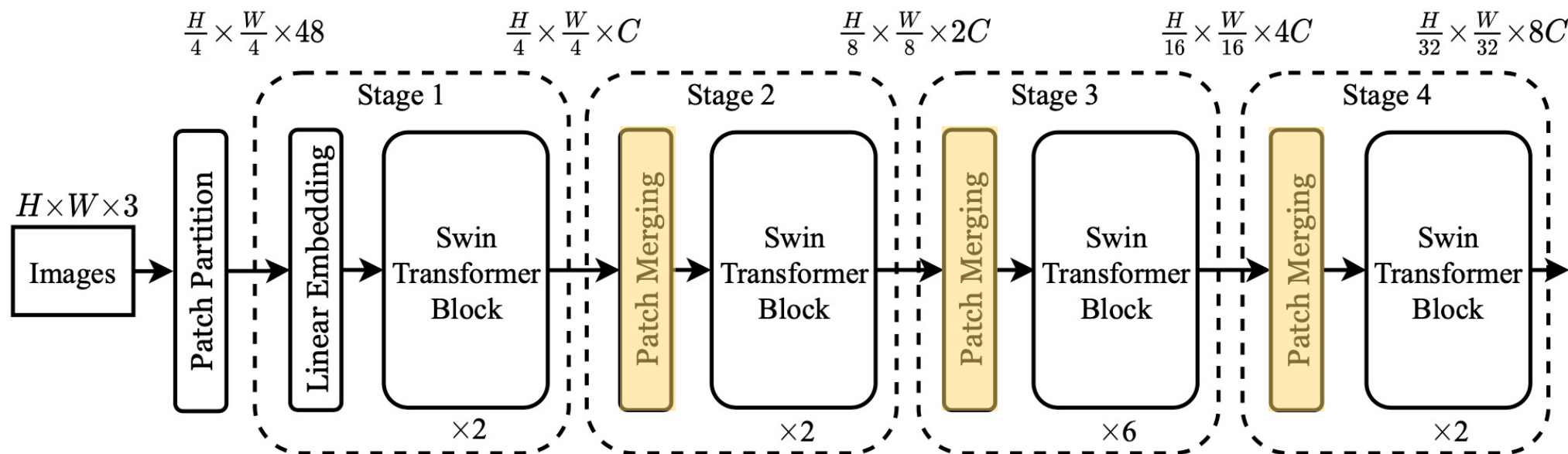
- Split an input RGB image into non-overlapping patches
- Treat each patch as a “token” ($4 \times 4 \times 3 = 48$), number of tokens ($\frac{H}{4} \times \frac{W}{4}$)
- Concatenate raw pixel RGB values as patch feature set

Linear Embedding

Applied on the raw-valued feature to project it to an arbitrary dimension, denoted as C

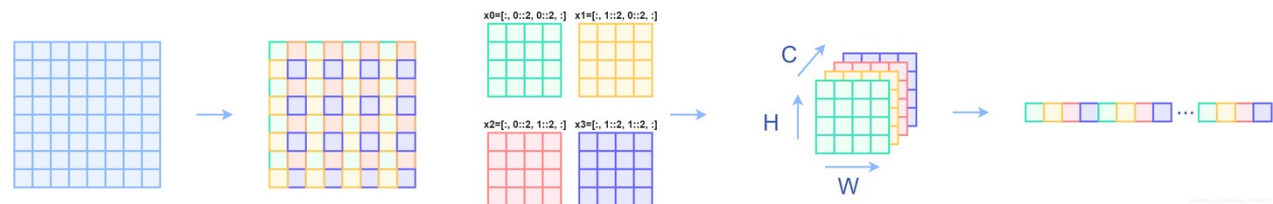


Swin Transformer

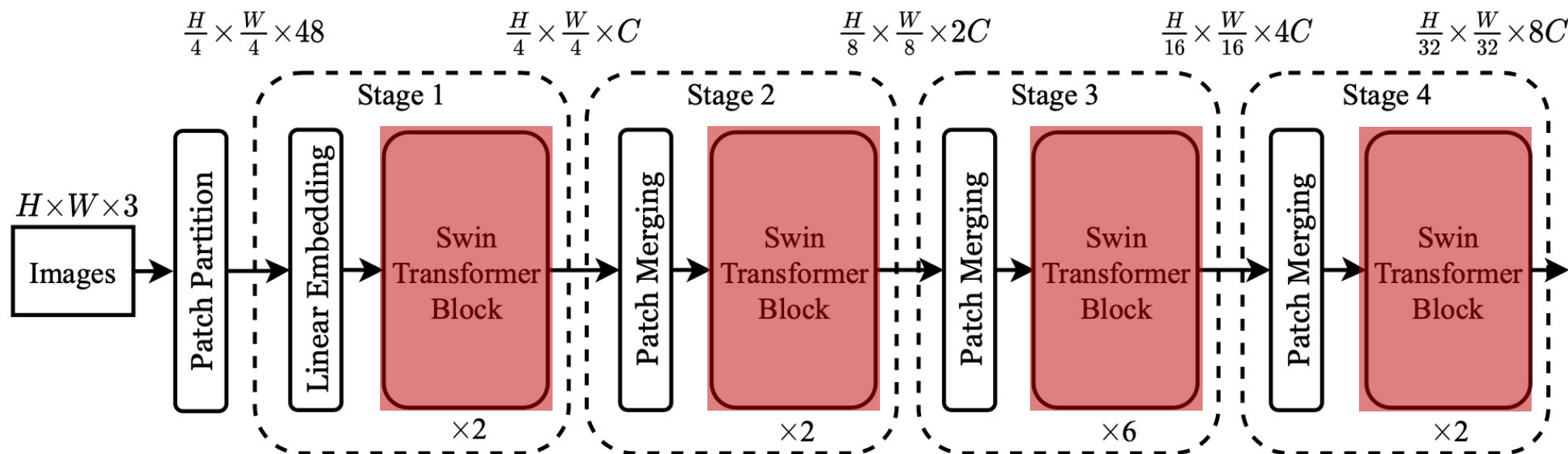


Patch Merging

- Reminiscent the pooling in CNN
- Reduce the dimension, form hierarchical representation
- Concatenate the features of each group of 2×2 neighboring patches
- Apply a linear layer on the $4C$ -dimensional concatenated features to get an output dimension of $2C$



Swin Transformer



Swin Transformer Block

With modified self-attention computation

Repeating the same process of Stage 2 to produce a hierarchical representation

Swin Transformer

Swin Transformer conducts **local self-attention**

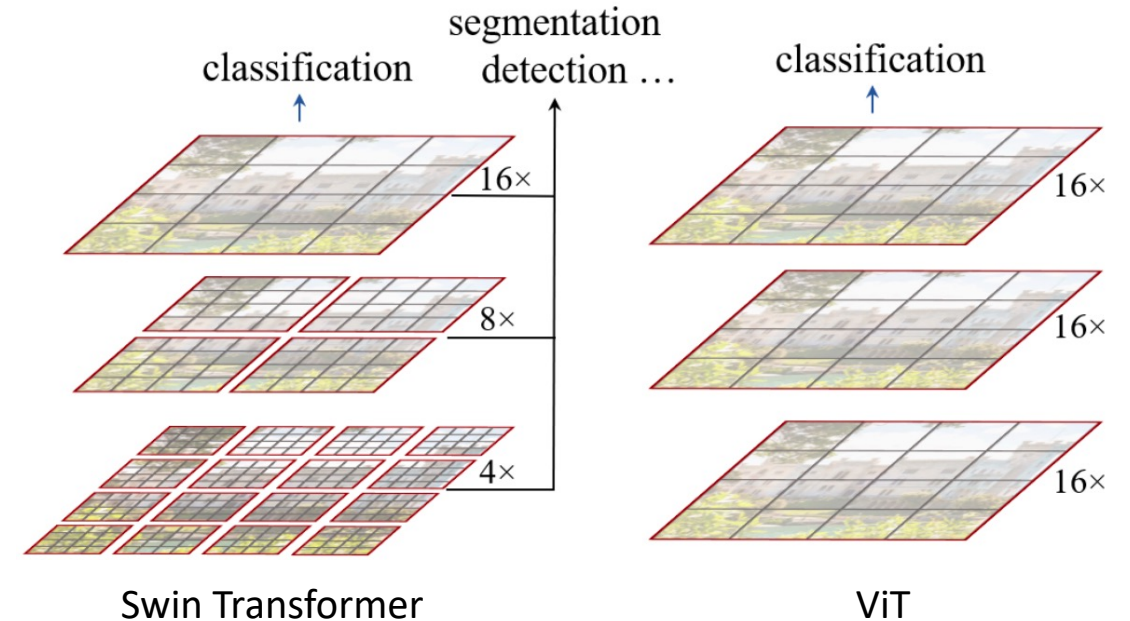
- Limiting self-attention computation to non-overlapping local windows
- The number of patches in each window is fixed
- Supposing each window contains $M \times M$ patches and there are $h \times w$ patches on an image

Conventional multi-head self attention (MSA), quadratic to patch number hw (omit SoftMax)

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C$$

Window-based self attention (W-MSA), linear to patch number hw (omit SoftMax)

$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC$$



Swin Transformer

Supposing each window contains $M \times M$ patches and there are $h \times w$ patches on an image.

For conventional multi-head self attention (MSA):

1. For each patch we need to compute the respective Q , K , and V , with $\mathbf{Q} = \mathbf{XW}^Q$, $\mathbf{K} = \mathbf{XW}^K$, and $\mathbf{V} = \mathbf{XW}^V$, where $\mathbf{X} \in \mathbb{R}^{hw \times C}$ and $\mathbf{W} \in \mathbb{R}^{C \times C}$. Thus, the complexity is $3hwC^2$
2. Compute $\mathbf{QK}^T$, and we know $\{\mathbf{Q}, \mathbf{K}, \mathbf{V}\} \in \mathbb{R}^{hw \times C}$. Thus, the complexity is $(hw)^2C$
3. Apply SoftMax and multiply with $\mathbf{V}$ to obtain $\mathbf{Z}$. Since $\mathbf{QK}^T \in \mathbb{R}^{hw \times hw}$, this operation takes $(hw)^2C$
4. The final output is obtained by multiplying $\mathbf{Z}$ with the output matrix $\mathbf{W}^Z$. The complexity is hwC^2

Hence, the final complexity is $\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C$ - quadratic to patch number hw

if $C = AB$ for an $n \times m$ matrix A and an $m \times p$ matrix B , then C is an $n \times p$ matrix with entries. This algorithm takes time $\Theta(nmp)$ (in asymptotic notation).

Swin Transformer

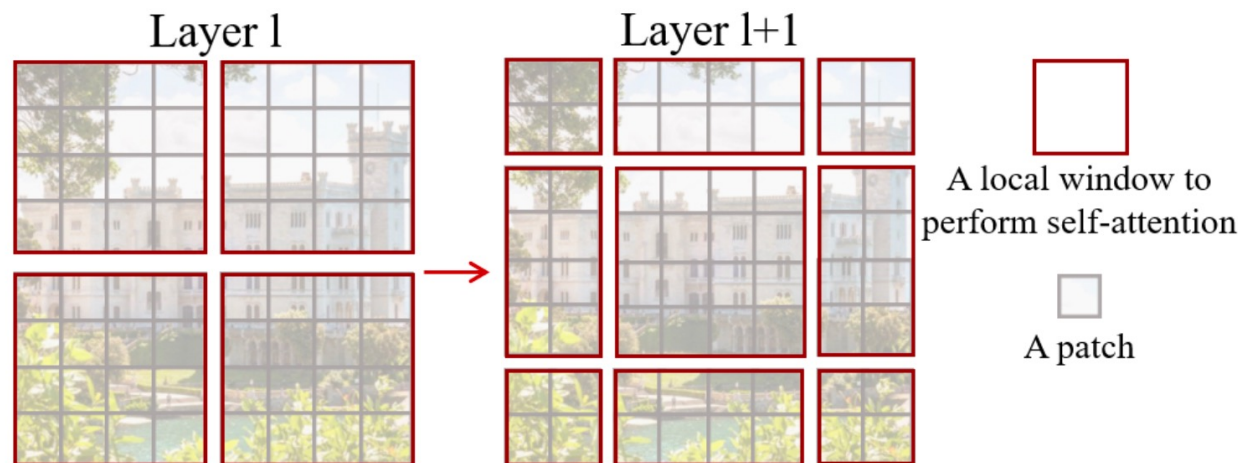
Window-based self attention (W-MSA) performs self-attention within windows, each of which contains $M \times M$ patches

1. [Same as MSA] the complexity is $3hwC^2$
 2. In W-MSA, there are $\frac{h}{M} \times \frac{w}{M}$ windows. In each window, the complexity of computing $\mathbf{QK}^T$ is $(M^2)^2 C$ and thus the total complexity is $\frac{h}{M} \times \frac{w}{M} (M^2)^2 C = M^2 hwC$
 3. Apply SoftMax and multiply with $\mathbf{V}$ to obtain $\mathbf{Z}$. Similar to aforementioned reason, this operation takes $M^2 hwC$
 4. [Same as MSA] the complexity is hwC^2
- Hence, the final complexity is $\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2 hwC$ – linear to patch number hw

Swin Transformer

Shifted window partitioning in successive blocks

- Enhance connections by bridging the windows of the preceding layer to improve modelling power
- Alternating between two partitioning configurations



A regular window partitioning scheme is adopted, and self-attention is computed within each window

The window partitioning is shifted, resulting in new windows. The self-attention computation in the new windows crosses the boundaries of the previous windows in preceding layer, providing connections among them

Swin Transformer

Successive Swin Transformer Blocks

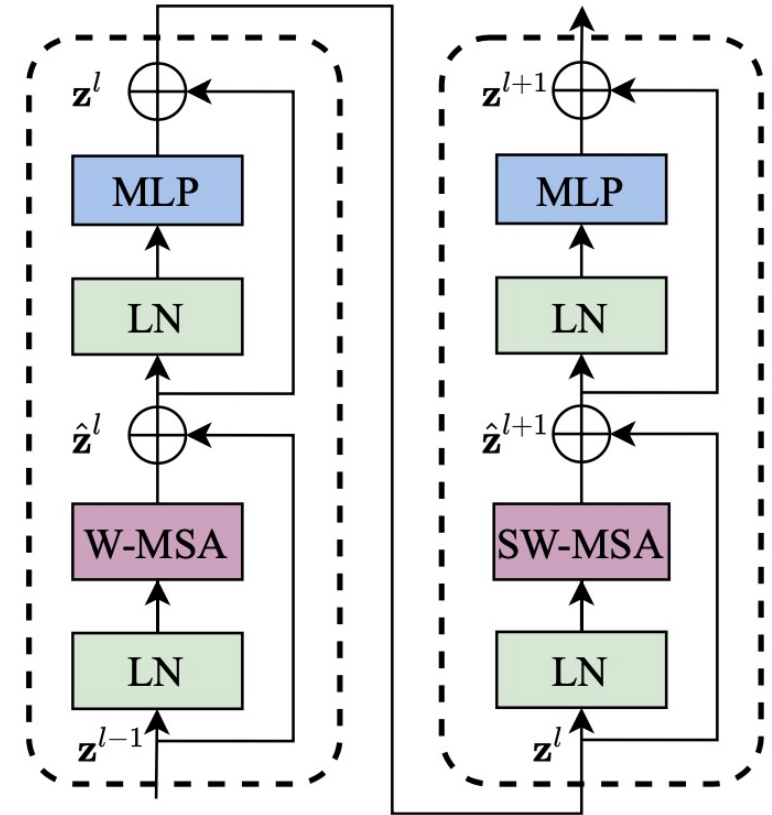
- The first block consists of a window-based MSA module (**W-MSA**), followed by a 2-layer MLP (with GELU)
- The second block consists of a shifted window based MSA module (**SW-MSA**), followed by a 2-layer MLP (with GELU)
- Layer normalization (LN) is applied before each MSA module and each MLP, and a residual connection is applied after each module.

$$\hat{\mathbf{z}}^l = \text{W-MSA}(\text{LN}(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1},$$

$$\mathbf{z}^l = \text{MLP}(\text{LN}(\hat{\mathbf{z}}^l)) + \hat{\mathbf{z}}^l,$$

$$\hat{\mathbf{z}}^{l+1} = \text{SW-MSA}(\text{LN}(\mathbf{z}^l)) + \mathbf{z}^l,$$

$$\mathbf{z}^{l+1} = \text{MLP}(\text{LN}(\hat{\mathbf{z}}^{l+1})) + \hat{\mathbf{z}}^{l+1},$$

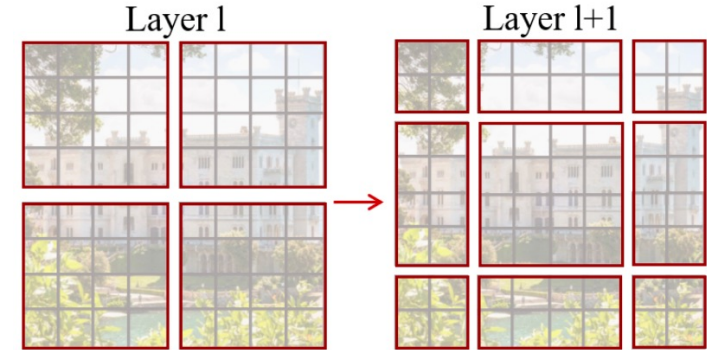


Two Successive Swin Transformer Blocks

Swin Transformer

Issues with shifted window partitioning

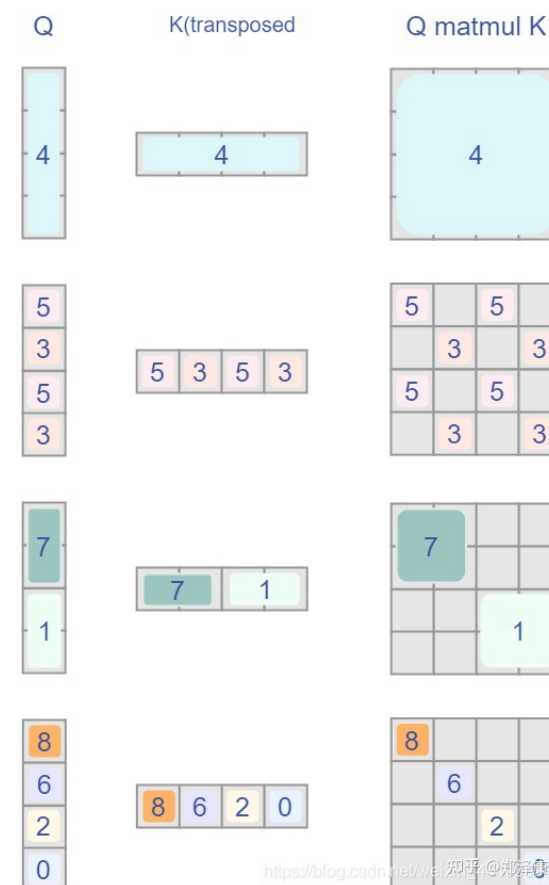
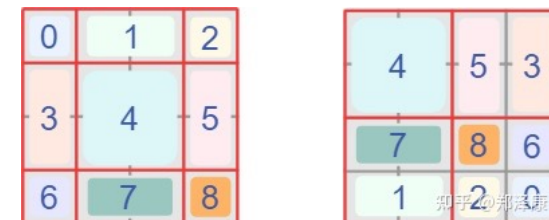
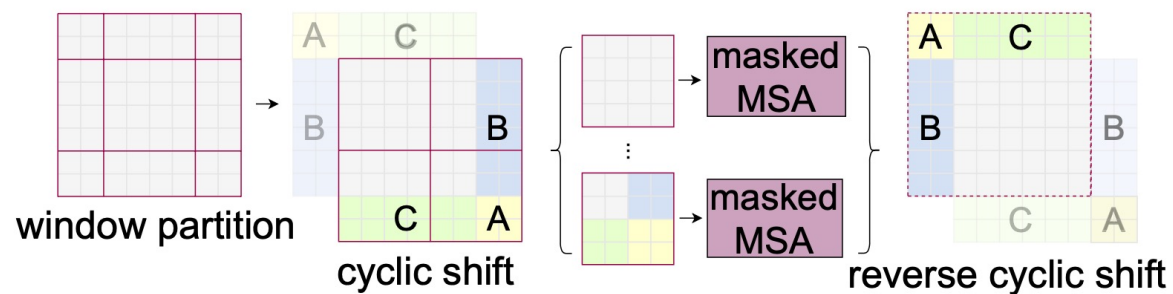
- Result in more windows, from $\left\lfloor \frac{h}{M} \right\rfloor \times \left\lfloor \frac{w}{M} \right\rfloor$ to $\left(\left\lfloor \frac{h}{M} \right\rfloor + 1 \right) \times \left(\left\lfloor \frac{w}{M} \right\rfloor + 1 \right)$ in the shifted configuration. For instance, from 2×2 to 3×3
- Some of the windows will be smaller than $M \times M$.
- Naïve solution: pad the smaller windows, but computation is 2.25 greater (for the case $2 \times 2 \rightarrow 3 \times 3$)



Swin Transformer

Efficient batch computation

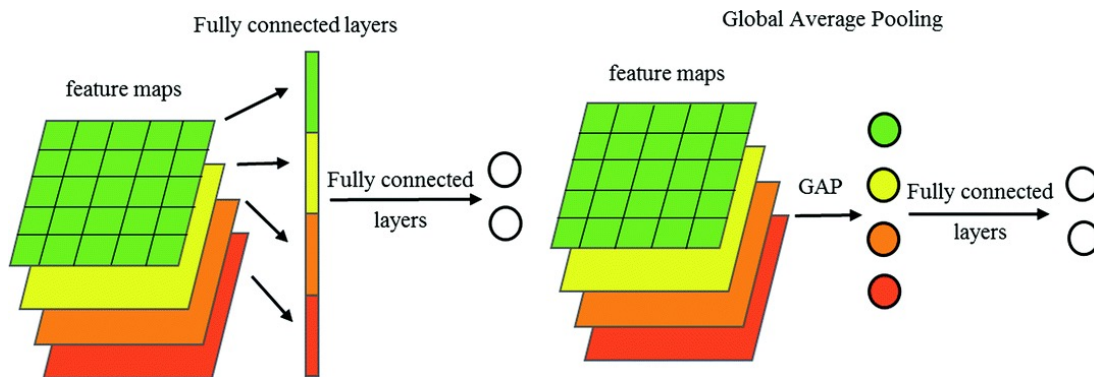
- The goal is to maintain the same number of batched windows as that of regular window partitioning
- Cyclic-shifting toward the top-left direction
- Using masked MSA to limit self-attention computation to within each sub-window



Swin Transformer

Image Classification

- The image classification is performed by applying a global average pooling layer on the output feature map of the last stage, followed by a linear classifier
- Found to be as accurate as using an additional class token as in ViT



(a) Regular ImageNet-1K trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [47]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [47]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [47]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [57]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [57]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [57]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [57]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [57]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [60]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [60]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [60]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.3
Swin-B	384 ²	88M	47.0G	84.7	84.2

(b) ImageNet-22K pre-trained models

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [37]	384 ²	388M	204.6G	-	84.4
R-152x4 [37]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [19]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [19]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.0
Swin-L	384 ²	197M	103.9G	42.1	86.4

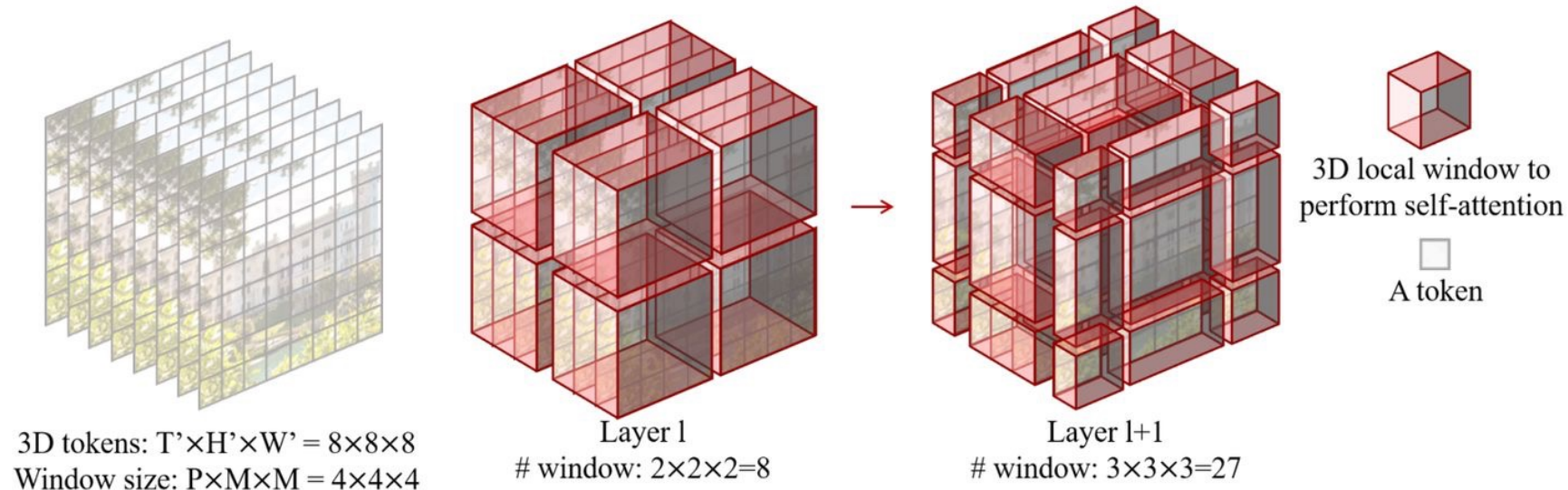
Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [65] and a V100 GPU, following [60].

Swin Transformer achieves a better speed-accuracy trade-off than state-of-the-art CNNs

Video Swin Transformer

Extension to spatiotemporal domain

The mechanism of shifted windows is reformulated to process spatiotemporal input

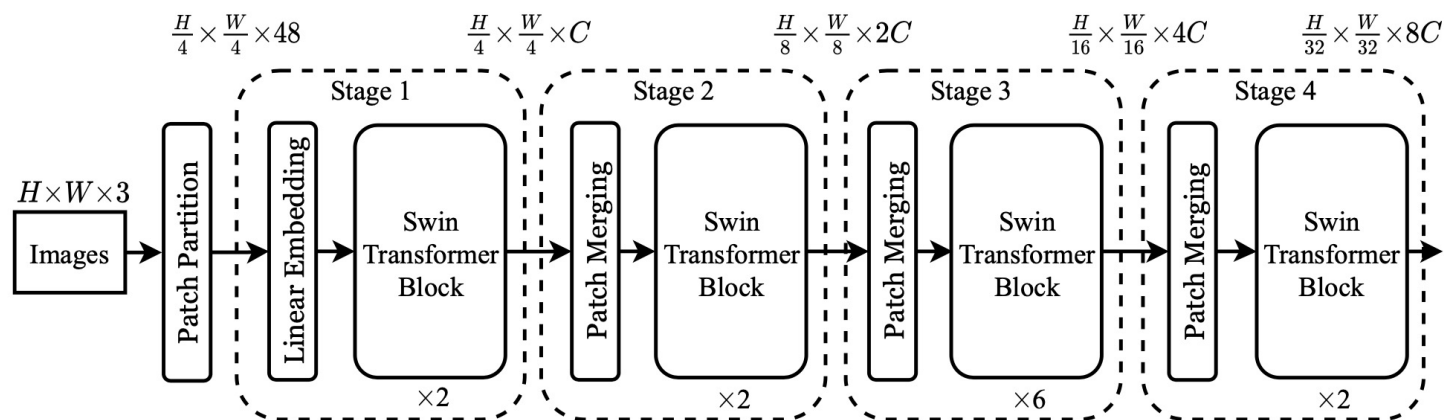


Video Swin Transformer

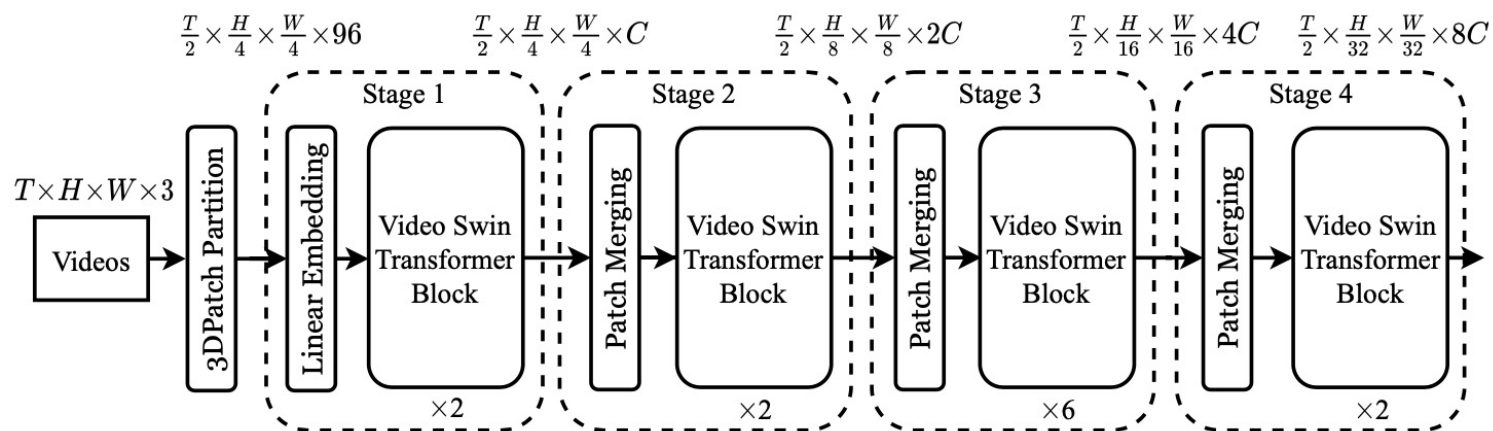
Extension to spatiotemporal domain

The architecture is also adjusted

Swin Transformer



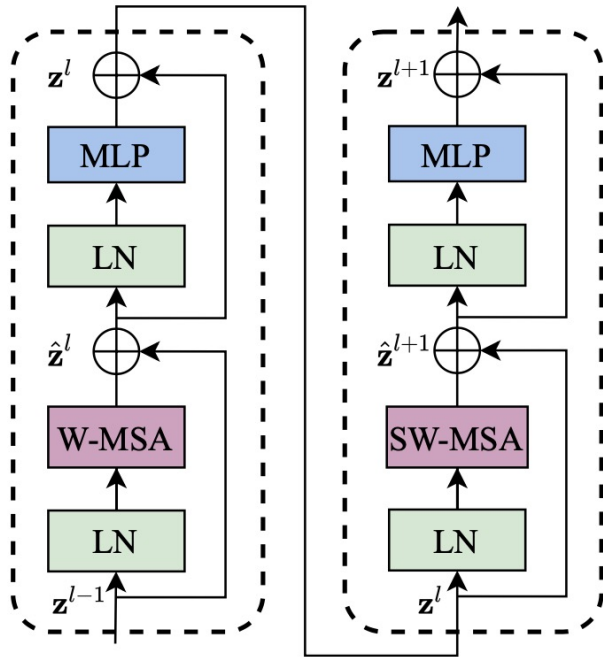
Video Swin Transformer



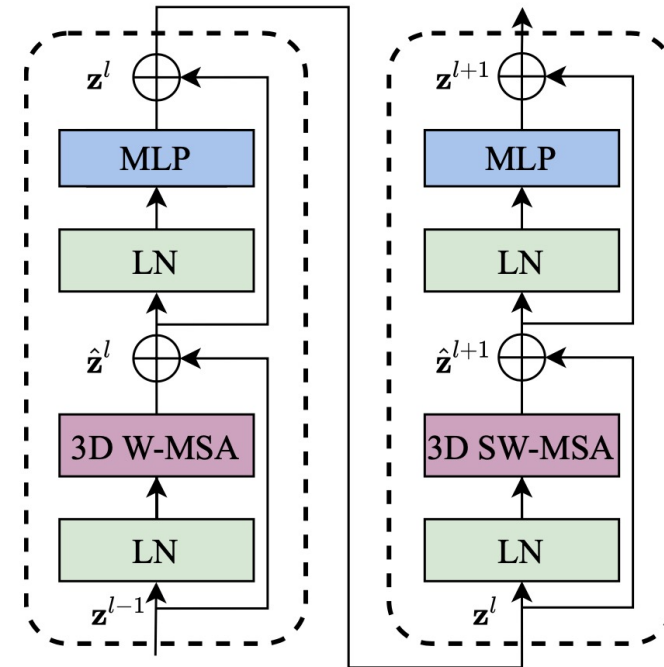
Video Swin Transformer

Extension to spatiotemporal domain

The architecture is also adjusted

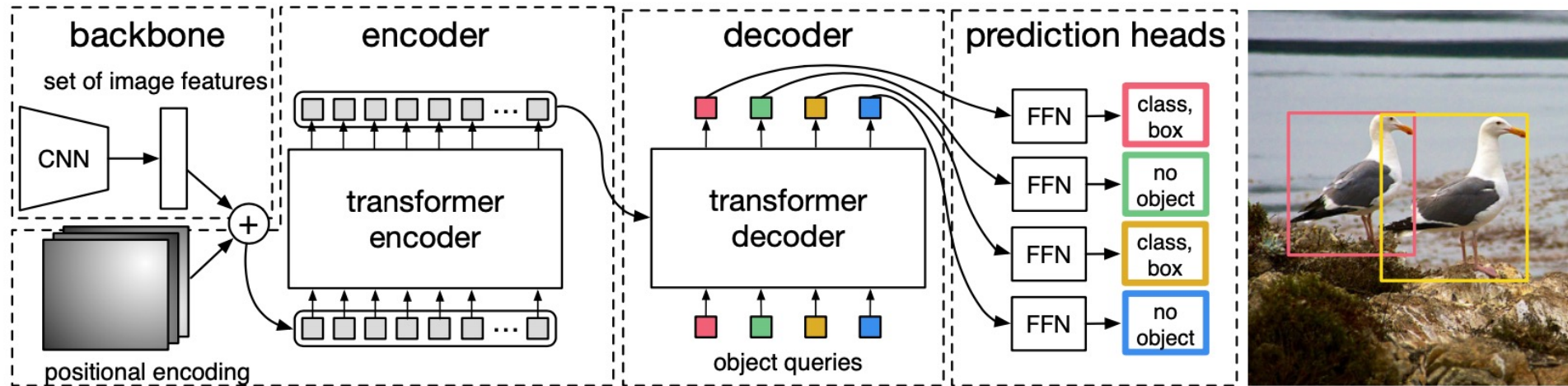


Swin Transformer



Video Swin Transformer

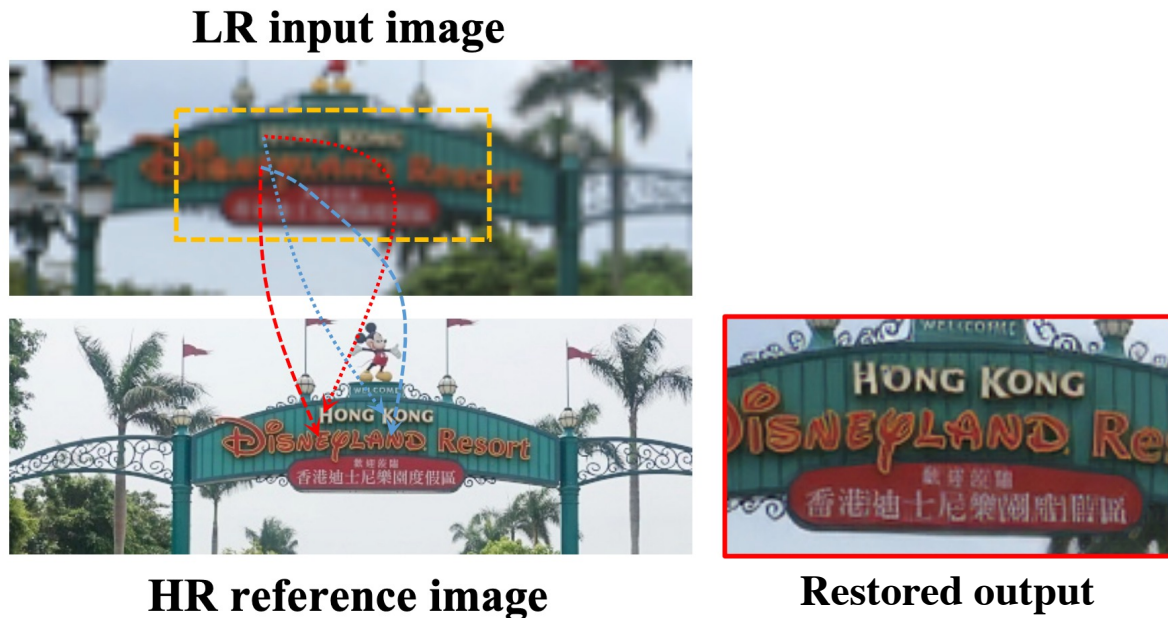
Applications of Transformer



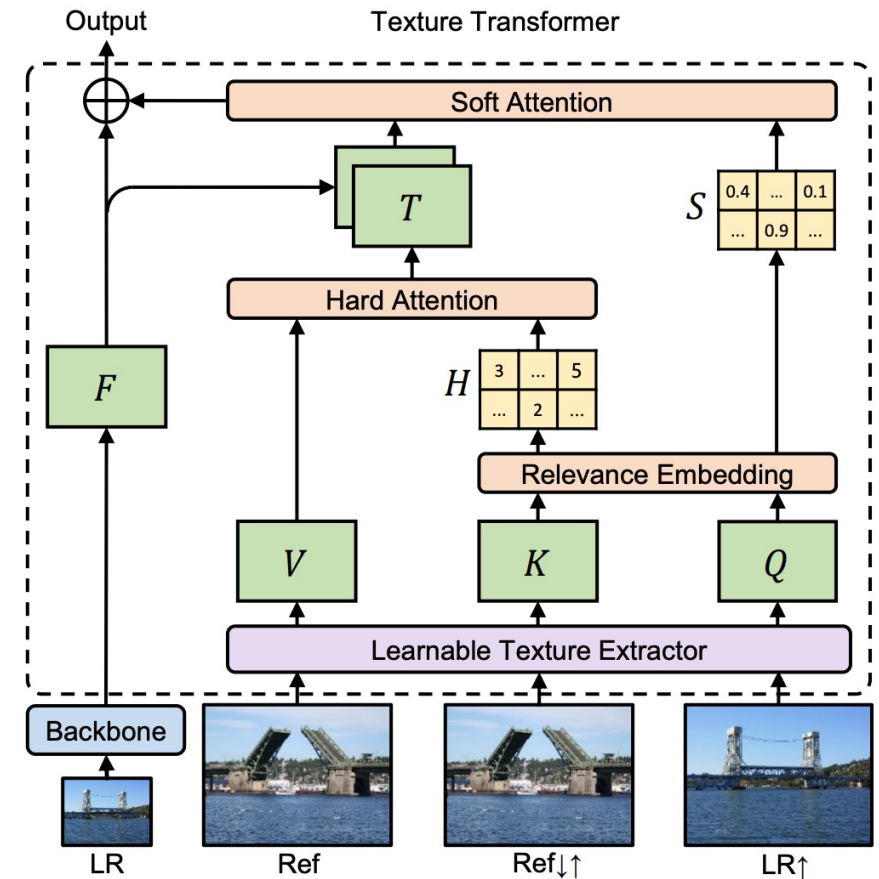
Set prediction problem: DETR simplifies the detection pipeline by dropping multiple hand-designed components that encode prior knowledge, like spatial anchors or non-maximal suppression.

- DETR uses a conventional CNN backbone to learn a 2D representation of an input image.
- The model flattens it and supplements it with a positional encoding before passing it into a transformer encoder.
- A transformer decoder then takes as input a small fixed number of learned positional embeddings (object queries), and additionally attends to the encoder output.
- Pass each output embedding of the decoder to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a "no object" class.

Applications of Transformer



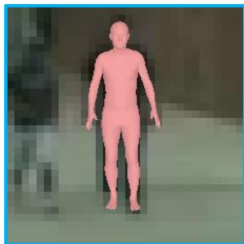
Super-resolves the LR image with the guidance of an additional high-resolution (HR) reference image. Textures of the HR reference image are transferred to provide more fine details for the LR image.



Applications of Transformer

3D human texture estimation from a single image

Input image



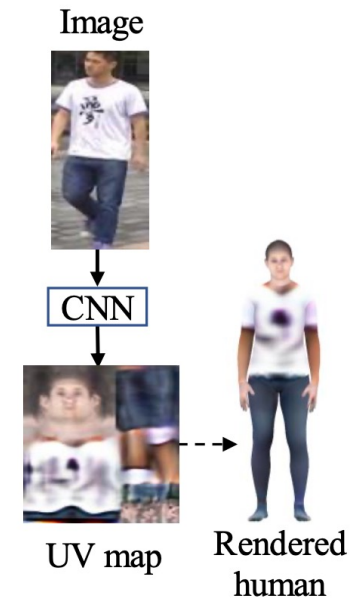
Applications of Transformer

3D human texture estimation from a single image

Most of existing methods CNNs to predict 3D human texture (i.e., a UV map) from the input image

The input and output **do not have strictly-aligned spatial correspondences** and may even have totally different shapes

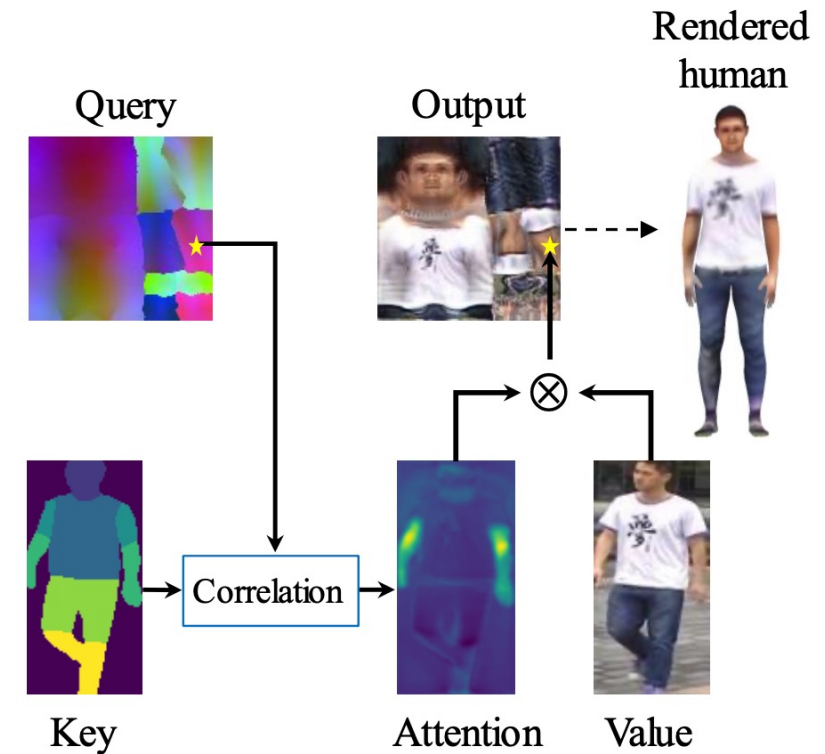
Convolution layers are by design **local operations** and inefficient in processing global information that is crucial in 3D human texture estimation



Applications of Transformer

3D human texture estimation from a single image

Transformer uses the **attention mechanism to more effectively exploit global information**, which leads to higher-quality 3D human texture estimation.



Query = color encoding map obtained from 3D coordinates of a standard 3D body mesh, and each element in the Query corresponds to a physical vertex

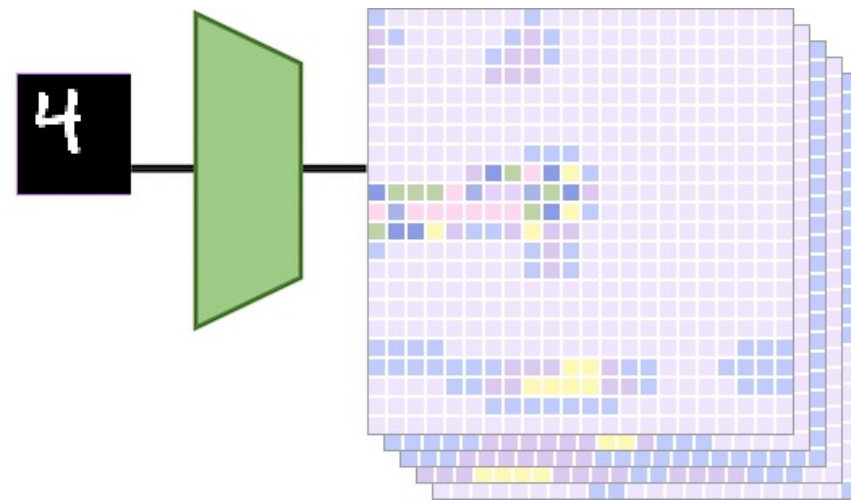
Pretraining of Transformers

Attain excellent results when pretrained on **large-scale** datasets (e.g., JFT-300M)

Lower accuracy than ResNet when it is trained on ImageNet

Lack inherent **inductive biases** inherent to CNN

- *Inductive biases are the characteristics of learning algorithms that influence their generalization behaviour, independent of data.*
- Translation equivariance, two-dimensional neighborhood structure, and locality
- And thus do not generalize well given insufficient training data



An equivariant mapping is a mapping which preserves the algebraic structure of a transformation.

A translation equivariant mapping is a mapping which, when the input is translated, leads to a translated mapping

Transformers vs. CNN

- Do Transformers act like convolutions?
- Do they act like convolutions, learning the same inductive biases from scratch?
- Or are they developing novel task representations?

Transformers vs. CNN

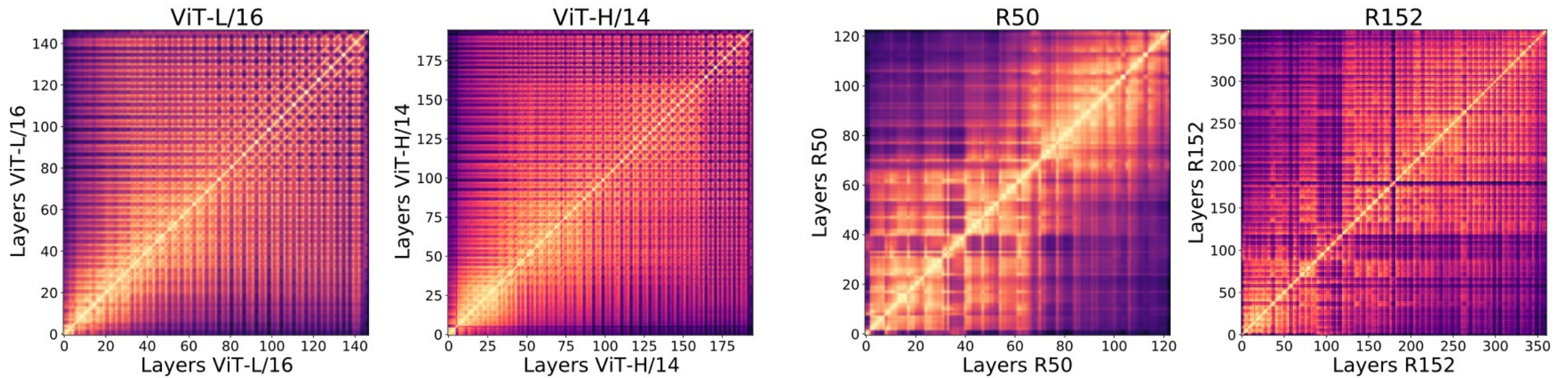
Tool: Centered kernel alignment (CKA) to compare the representations (activations) between all pairs of layers - measure meaningful similarities between representations of higher dimension than the number of data points

$$\text{CKA}(\mathbf{K}, \mathbf{L}) = \frac{\text{HSIC}(\mathbf{K}, \mathbf{L})}{\sqrt{\text{HSIC}(\mathbf{K}, \mathbf{K})\text{HSIC}(\mathbf{L}, \mathbf{L})}}$$

Gram matrices for the two layers

Hilbert-Schmidt Independence Criterion

Transformers vs. CNN



ViT having more uniform representations, with greater similarity between lower and higher layers.

Transformers vs. CNN

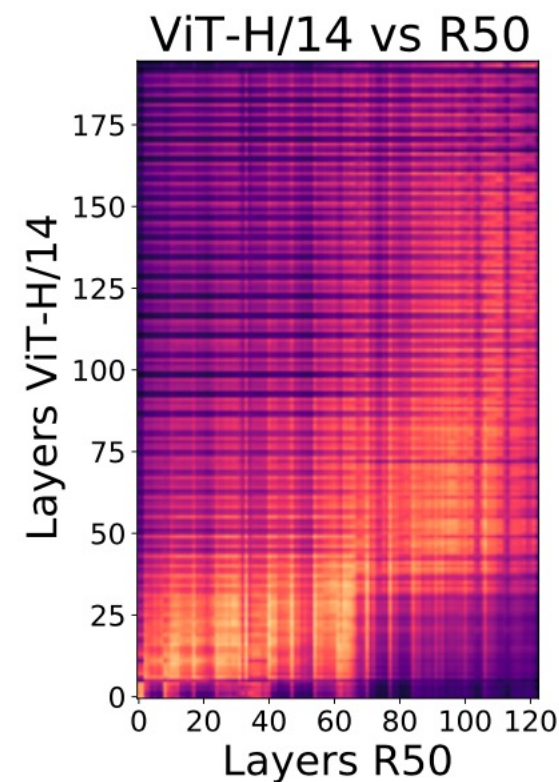
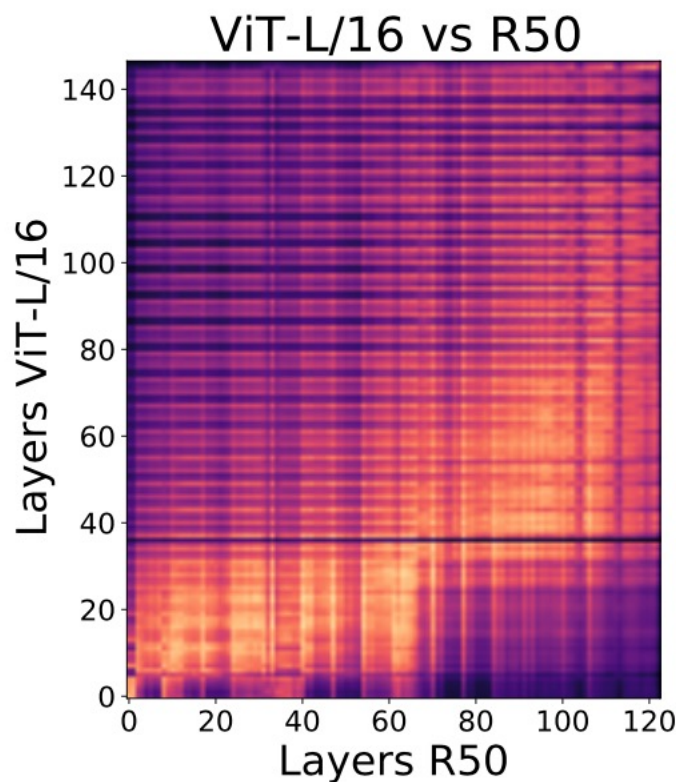
Cross-model comparisons

The lower half of 60 ResNet layers are similar to approximately the lowest quarter of ViT layers.

In particular, many more lower layers in the ResNet are needed to compute similar representations to the lower layers of ViT.

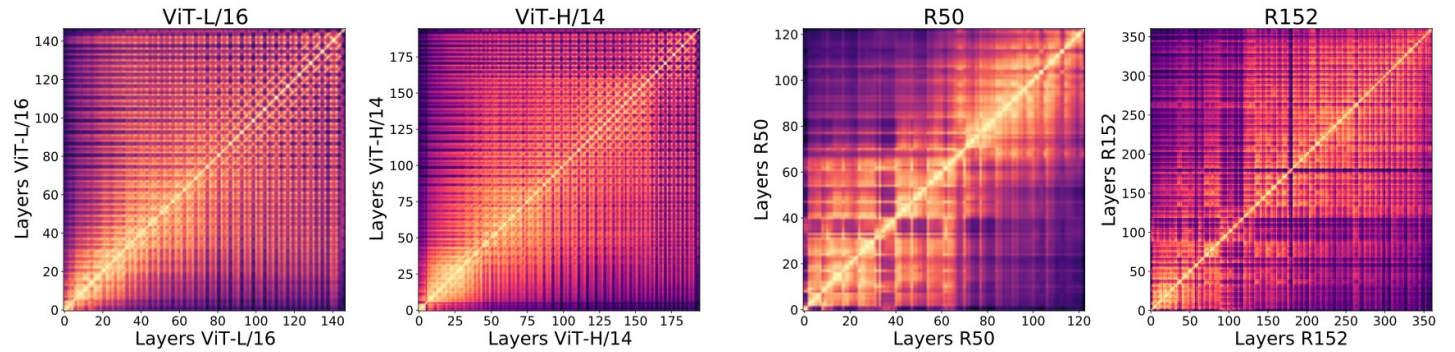
The top half of the ResNet is approximately similar to the next third of the ViT layers.

The final third of ViT layers is less similar to all ResNet layers

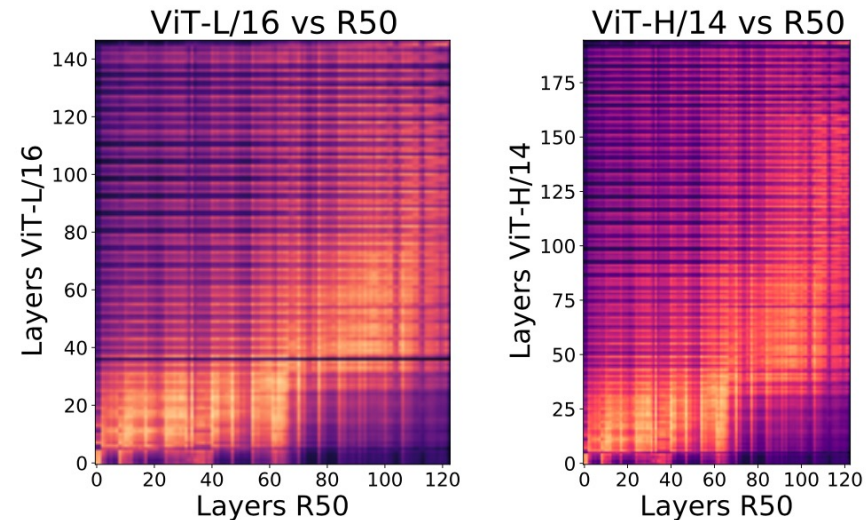


Transformers vs. CNN

- (i) ViT lower layers compute representations in a different way to lower layers in the ResNet
- (ii) ViT also more strongly propagates representations between lower and higher layers
- (iii) The highest layers of ViT have quite different representations to ResNet



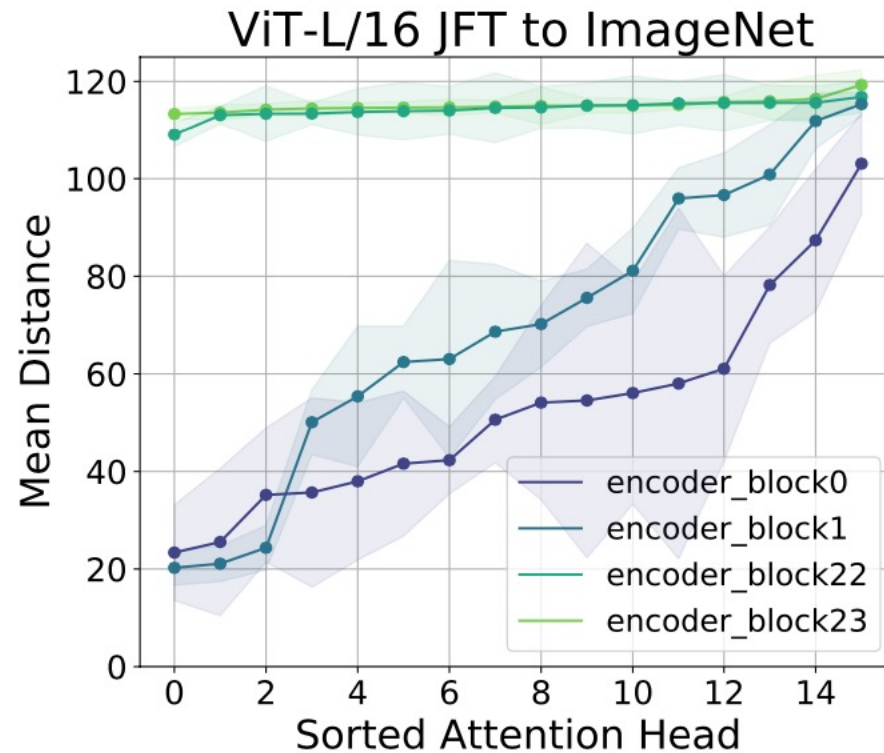
Within model



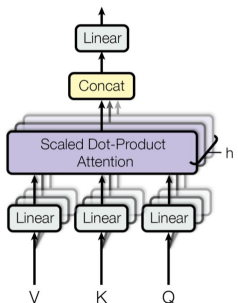
Cross model

Transformers vs. CNN

With large-scale pre-training



Average distance between the query patch position and the locations it attends to



Heads sorted by their average distance.

Two highest layers in the ViT

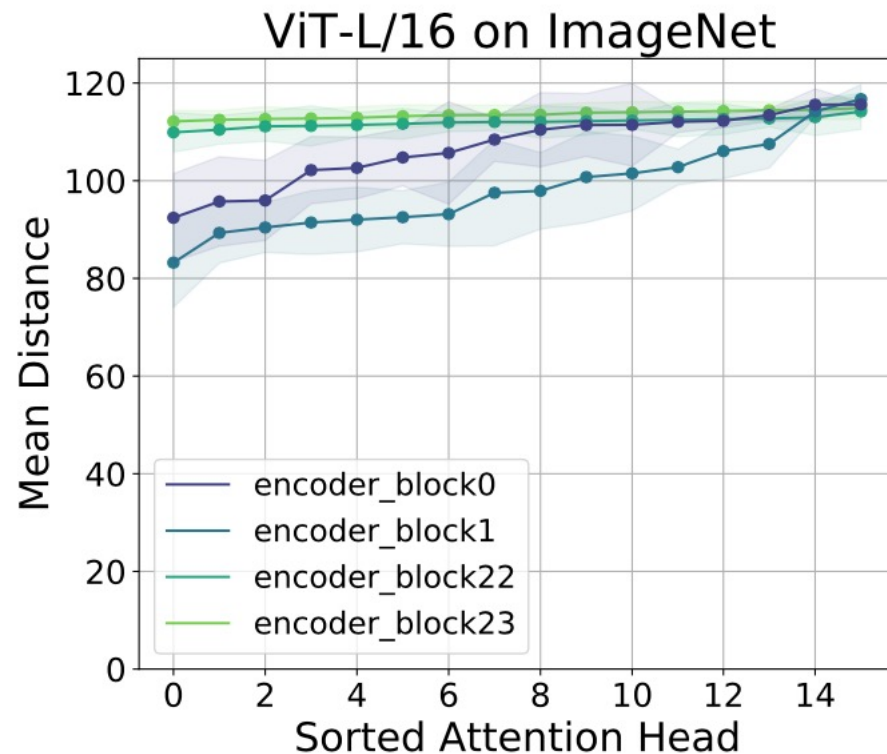
At higher layers, all self-attention heads are global.

Two lowest layers in the ViT

Self-attention layers have a mix of local heads (small distances) and global heads (large distances). This is in contrast to CNNs, which are hardcoded to attend only locally in the lower layers.

Transformers vs. CNN

With NO large-scale pre-training (just training on ImageNet) – much lower performance



Average distance between the query patch position and the locations it attends to

Heads sorted by their average attention.

Two highest layers in the ViT

At higher layers, all self-attention heads are global.

Two lowest layers in the ViT

ViT does not learn to attend locally in earlier layers!!!

Using local information early on for image tasks (which is hardcoded into CNN architectures) is important for strong performance

Further Reading

- Research trend: Hybrid model (CNN + Transformer)
- Before Vision Transformer (ViT) → CNN + Attention
 - Non-local Neural Networks, CVPR 2018
 - GCNet: Non-local Networks Meet Squeeze-Excitation Networks and Beyond, ICCV 2019
 -
- After ViT → Transformer + CNN
 - Swin Transformer: Hierarchical Vision Transformer using Shifted Windows, ICCV 2021
 - Incorporating Convolution Designs into Visual Transformers, ICCV 2021 ([Locally-enhanced Feedforward](#))
 - Pyramid Vision Transformer: A Versatile Backbone for Dense Prediction without Convolutions, ICCV 2021
 - Tokens-to-Token ViT: Training Vision Transformers from Scratch on ImageNet, ICCV 2021 ([+ Dense connection\(DenseNet\)](#), [Shadow-wide\(Wide-ResNet\)](#), [Squeeze-and-Excitation](#), [Split heads \(ResNeXt\)](#), [Ghost operator\(GhostNet\)](#))
 - LeViT: a Vision Transformer in ConvNet's Clothing for Faster Inference, arXiv 2021.04
 - Early Convolutions Help Transformers See Better, arXiv 2021.06 ([Replace early transformer layers with CNN](#))
 - XCiT: Cross-Covariance Image Transformers, arXiv 2021.06 ([local patch interaction](#))
 - ViTAE: Vision Transformer Advanced by Exploring Intrinsic Inductive Bias, arXiv 2021.06
 - CrossFormer: A Versatile Vision Transformer Based on Cross-scale Attention, arXiv 2021.08 ([short distance attention](#))
 - Evo-ViT: Slow-Fast Token Evolution for Dynamic Vision Transformer arXiv 2021.08 ([local spatial prior](#))
 -